

Oriole: Adaptively Secure Partially Non-Interactive Threshold Signatures from Lattices

Kaijie Jiang¹ , Hoeteck Wee², and Chenzhi Zhu² 

¹ Institute for Advanced Study, Tsinghua University, Beijing, China

² NTT Research, California, US

Abstract. We present the first lattice-based, partially non-interactive threshold signature scheme that tolerates the adaptive corruption of up to $T - 1$ signers, where T is the signing threshold. Our construction relies on the MSIS and MLWE assumptions, and has two rounds, of which only the second is message-dependent. We substantially improve upon prior adaptively secure lattice-based schemes (CRYPTO '24 and EUROCRYPT '26), which require at least two message-dependent rounds. Compared to prior lattice-based partially non-interactive assumptions (CRYPTO '24, S&P '25, CRYPTO '25), we achieve better communication complexity in addition to stronger security guarantees.

1 Introduction

In a T -out-of- N threshold signature [Des88, DF90], a group of N parties hold shares of a signing key so that any subset of at least T out of N parties can produce a valid signature, while any adversary corrupting fewer than T of these parties cannot forge new signatures. Threshold signatures play a crucial role in securing decentralized systems, distributed key management, and fault-tolerant cryptographic protocols. They are widely deployed in practice, particularly for securing digital wallets and other high-value assets. Ongoing IETF [CKGW22] and NIST [Natnt] standardization efforts further underscore their growing practical importance. Thanks to a substantial and fruitful body of recent work, we now have efficient threshold signature schemes from pairings [Bol03, BL22, DR24], RSA [DDFY94, GRJK07, Sho00], pairing-free groups [Mak22, BLT⁺24, KRT24, BDLR25b, Che25a, CKK⁺25, BDLR25a, GCRS25, Che25b, CKK⁺25, BCL⁺26], and from lattices [BGGK17, BGG⁺18, ASY22, GKS24, DKM⁺24, EKT24, KRT24, CATZ24, ZT25, BKL⁺25, dPENP25, PKN⁺25, BCdP⁺25, BdCE⁺25, JTWZ26].

As deployment settings become increasingly adversarial and asynchronous, two properties emerge as central: minimal online interaction and strong adaptive security against adversaries that may choose which parties to corrupt as the protocol evolves. In particular, partially non-interactive signing—where a single round of message-independent preprocessing enables signature generation in a single online round—substantially improves scalability, latency, and robustness. However, among existing constructions, only [BL22, DR24, CKK⁺25, BCL⁺26, Che26] simultaneously achieve partial non-interactivity and adaptive security, and they all rely on classical group-based assumptions. Consequently, no known scheme provides these guarantees under post-quantum assumptions, leaving a fundamental gap in the lattice setting. Addressing this gap is particularly timely, as NIST has explicitly noted the importance of both post-quantum and adaptive security in potential threshold signature standards.

This work. In this work, we close this gap with Oriole, the first lattice-based threshold signature scheme that simultaneously achieves partially non-interactive signing and strong adaptive security. The security of our scheme is based on the Module Learning With Errors (MLWE) and Module Short Integer Solution (MSIS) assumptions in the random oracle model. This is a significant improvement over the only existing adaptively secure lattice-based threshold signature [KRT24, JTWZ26], which requires at least two message-dependent rounds. In Figure 1, we position our scheme in the broader context of the most closely related schemes.

For 128-bit security, our parameter estimation indicates that Oriole produces signatures of approximately 110KB, which are even smaller than the 160KB signatures reported for Tweed [JTWZ26], a lattice-based two-round adaptively secure scheme. While our signatures are larger than those of prior lattice-based partially non-interactive schemes achieving only static security [EKT24, ZT25, BKL⁺25], which obtain sizes in the range of 10–30KB, our

Scheme	Rounds Corruptions	
[BGGK17, BGG ⁺ 18, ASY22]	1	static
[EY24]	1	part. ad.
[GKS24, GTJ ⁺ 26]	0 + 2	static
[DKM ⁺ 24]	0 + 3	static
[EKT24, CATZ24, ZT25, BKL ⁺ 25]	1 + 1	static
[KRT24]	2 + 3	adaptive
[FNR25, ENP24]	2 + 1	static
[JTWZ26]	0 + 2	adaptive
This work	1 + 1	adaptive

Scheme	Rounds	Assumption
Dazzle [Che25a]	0 + 2	DDH
HBTS-Mask [Che25b]	0 + 2	DL
HBTS2[DDH]-Mask [Che26]	1 + 1	DDH
FROST [CKK ⁺ 25]	1 + 1	AOMDL + LDVR
ms-FROST [BCL ⁺ 26]	1 + 1	AOMDL + AGM
Tweed [JTWZ26]	0 + 2	MLWE + MSIS
This work	1 + 1	MLWE + MSIS

Fig. 1. Summary of T -out-of- N threshold signatures. The notation $X + Y$ denotes the number of offline (i.e., message independent) and online (i.e., message dependent) rounds, respectively. On the left, we list all existing constructions from lattices. On the right, we list threshold signatures proved to be adaptively secure from pairing-free groups and lattices, with at most two rounds. We only consider schemes that are stateless and without erasures, and for which optimal corruptions (up $T - 1$ parties) are tolerated. Also, all schemes additionally rely on the random oracle model.

scheme achieves significantly better communication complexity, requiring roughly about 150KB per signer, compared to 280KB in [EKT24] (or 760KB based on analysis under the standard lattice assumptions [ZT25]) and 610KB in [BKL⁺25]. Still, we emphasize that our primary contribution lies in introducing new techniques for achieving adaptive security along with partial non-interactivity, and leave the improvement of parameters to future work.

Our approach. At a conceptual level, a key obstacle in constructing lattice-based, partially non-interactive threshold signatures with adaptive security lies in the interaction between proof techniques and corruption models. Existing partially non-interactive schemes [EKT24, CATZ24, ZT25, BKL⁺25] rely on rewinding-based security proofs, which are ill-suited for adaptive security. The recent adaptively secure schemes KRT [KRT24] and Tweed [JTWZ26] circumvent rewinding by adapting techniques originally developed for DL/DDH-based schemes in pairing-free groups (e.g. Dazzle [Che25a]) to lattices. However, the pairing-free constructions that additionally achieve partial non-interactivity [CKK⁺25, BCL⁺26] rely on stronger assumptions that do not naturally carry over to lattice settings under standard assumptions.

Motivated by these challenges, we revisit the design space of lattice signatures. The threshold schemes we just described are based on Fiat–Shamir-style challenge–response protocols. In contrast, lattice-based hash-and-sign signatures admit non-rewinding proofs [GPV08, MP12, CGM19, YJW23, EEN⁺24], providing a more promising foundation for adaptive security. However, current threshold hash-and-sign schemes [FNR25, ENP24] achieve only static security and require three rounds, with a single message-dependent round.

Our approach builds on this alternative foundation. We first show that the hash-and-sign framework can be adapted to obtain a partially non-interactive threshold scheme with static security and without rewinding. We then develop new simulation techniques to prove adaptive security.

Concurrent work. In an independent and concurrent work [Che26], Chen presents a partially non-interactive threshold signature scheme based on DDH in pairing-free groups; the scheme also tolerates adaptive corruption of up to $T - 1$ signers. The scheme is based on Fiat–Shamir-style challenge–response protocols.

2 Technical Overview

We fix a ring $\mathcal{R}$ (either $\mathbb{Z}$ or $\mathbb{Z}[X]/(X^d + 1)$) and let $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$.

Hash-and-sign from SIS. The starting point of our construction is the (salted) GPV signature [GPV08]:

- The public key consists of a matrix $\mathbf{A} \in \mathcal{R}_q^{n \times m}$, along with a hash function $H_1 : \{0, 1\}^* \rightarrow \mathcal{R}_q^n$, and the secret key is a trapdoor $\mathbf{T}$ of $\mathbf{A}$.

- To sign a message μ , the signer first samples $\text{salt} \leftarrow \{0, 1\}^\lambda$, and then uses the trapdoor to sample a gaussian $\mathbf{z} \in \mathcal{R}_q^m$ such that $\mathbf{A}\mathbf{z} = H_1(\mu, \text{salt})$. It outputs $(\mathbf{z}, \text{salt})$ as the signature.
- The verifier checks that $\|\mathbf{z}\|$ is small and that $\mathbf{A}\mathbf{z} = H_1(\mu, \text{salt})$.

The scheme satisfies (strong) unforgeability under (module) SIS. The security proof relies on the fact that we can simulate signatures on μ without a trapdoor for $\mathbf{A}$ by sampling a gaussian $\mathbf{z} \in \mathcal{R}_q^m$ along with a random salt and programming $H_1(\mu, \text{salt}) = \mathbf{A}\mathbf{z}$. Following [MP12, EEN⁺24], given $\mathbf{A}$ with trapdoor $\mathbf{T}$ such that $\mathbf{A}\mathbf{T} = \mathbf{G}$ (where $\mathbf{G}$ is the gadget matrix), we compute the pre-image $\mathbf{z}$ of $H_1(\mu, \text{salt})$ as follows:

$$\mathbf{z} = \mathbf{r} + \mathbf{T} \cdot \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{A}\mathbf{r}) . \quad (1)$$

where $\mathbf{r}$ is random gaussian, $\mathbf{G}^{-1}(\cdot)$ is deterministic, and we use noise flooding with $\|\mathbf{r}\| \gg \|\mathbf{T}\|$ to ensure that $\mathbf{z}$ does not leak $\mathbf{T}$.

Our threshold signature. In the threshold scheme, signing is distributed across a subset SS of T out of N signers. We follow the same high-level approach as prior lattice-based schemes:

- Key generation assigns each party a Shamir T -out-of- N share $\mathbf{T}_i$ of the signing key $\mathbf{T}$ (so that any subset of $T - 1$ shares are uniformly random matrices over $\mathcal{R}_q$);
- To sign a message μ , each party $i \in \text{SS}$ samples a gaussian $\mathbf{r}_i$ and derives a partial signature $\mathbf{z}_i$ such that $\mathbf{r} = \sum_{i \in \text{SS}} \mathbf{r}_i$, $\mathbf{z} = \sum_{i \in \text{SS}} \mathbf{z}_i$ satisfy (1).

Concretely, signing is divided into two steps $\text{Sign}_1, \text{Sign}_2$ that take place over two rounds, where the first round is message-independent:

$\text{Sign}_1(i, \mathbf{T}_i)$: output $\mathbf{w}_i = \mathbf{A}\mathbf{r}_i$.

$\text{Sign}_2(i, \mathbf{T}_i, \mathbf{r}_i, \mu, \text{SS}, \{(j, \mathbf{w}_j)\}_{j \in \text{SS}})$:

$$\begin{aligned} \mathbf{w} &= \sum_{j \in \text{SS}} \mathbf{w}_j . \\ \text{salt} &= H_2(\mu, \text{SS}, \{(j, \mathbf{w}_j)\}_{j \in \text{SS}}) . \\ \text{output } \mathbf{z}_i &= \mathbf{r}_i + L_{\text{SS}, i} \cdot \mathbf{T}_i \cdot \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) . \end{aligned}$$

The final signature is given by $(\sum_{i \in \text{SS}} \mathbf{z}_i, \text{salt})$. Correctness follows readily from $\mathbf{T} = \sum_{i \in \text{SS}} L_{\text{SS}, i} \cdot \mathbf{T}_i$, so that $\mathbf{r} = \sum_{i \in \text{SS}} \mathbf{r}_i$, $\mathbf{z} = \sum_{i \in \text{SS}} \mathbf{z}_i$ indeed satisfy (1). We make one more modification to the scheme so far for the proof of security, where-in we incorporate additional masking following [EKT24]:

- During key generation, party i additionally receives a masking key mk_i that allows the party given SS and a salt $\text{salt} \in \{0, 1\}^*$, derive a mask $\text{mask}_{i, \text{SS}, \text{salt}}$, which is a (pseudo)random vector over $\mathcal{R}_q$ subject to the constraint $\sum_{i \in \text{SS}} \text{mask}_{i, \text{SS}, \text{salt}} = \mathbf{0}$.
- During signing, party i computes $\mathbf{w}_i, \mathbf{z}_i$ as before, but outputs $\mathbf{z}'_i = \mathbf{z}_i + \text{mask}_{i, \text{SS}, \text{salt}}$ instead of $\mathbf{z}_i$.

The final signature is given by $(\sum_{i \in \text{SS}} \mathbf{z}'_i, \text{salt})$. Correctness follows as before since $\sum_{i \in \text{SS}} \mathbf{z}'_i = \sum_{i \in \text{SS}} \mathbf{z}_i$.

Comparison with prior schemes. A crucial distinction from prior lattice-based threshold hash-and-sign schemes [ENP24, FNR25] is that we derive the salt from $\{(j, \mathbf{w}_j)\}_{j \in \text{SS}}$ and not $\mathbf{w} = \sum_{j \in \text{SS}} \mathbf{w}_j$. This ensures that an adversary cannot substantially bias the distribution of the salt. In particular, an adversary can bias $\mathbf{w}$ in our protocol (e.g., set $\mathbf{w} = \mathbf{0}$) by choosing its message in the first round as a function of the messages from the honest parties. If the salt is derived from $(\mu, \mathbf{w})$ as is the case in prior schemes, this would allow the adversary to bias the salt and thus $H_1(\mu, \text{salt})$. Such an attack strategy is avoided in [ENP24] by using distributed key generation to sample $\mathbf{w}$, and in [FNR25] by using commit-then-open for $\mathbf{w}_i$; both of these work-arounds incur at least an extra round of communication. Another recent work [ALLW25] describes a non-interactive threshold variant of GPV signatures using completely different techniques, but the final signature size grows with the threshold (in particular, the dimensions of the matrix $\mathbf{A}$ underlying the GPV signature grows linearly with T).

Overview of static security. As a warm-up, we proceed to provide a proof overview for static security, where the bulk of the work lies in simulating the view of the adversary given only $\mathbf{A}$. The static adversary first specifies a set $\text{CS} \subseteq [N]$ of corrupted parties, where $|\text{CS}| < T$, and then learns $\{\mathbf{T}_i\}_{i \in \text{CS}}$, which we can simulate using uniformly random matrices over $\mathcal{R}_q$. Next, we need to simulate the messages from the honest parties:

- For the first-round message $\mathbf{w}_i$, we proceed exactly as in Sign_1 and computes $\mathbf{w}_i = \mathbf{A}\mathbf{r}_i$. (Sampling a random $\mathbf{w}_i$ also works for static security.)
- For the second-round message, thanks to the masks, the collection of partial signatures $\{\mathbf{z}'_i\}_{i \in \text{SS} \setminus \text{CS}}$ only leak their sum. As such, it suffices to simulate given $(\mu, \text{SS}, \{(j, \mathbf{w}_j)\}_{j \in \text{SS}})$, the partial sum $\sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{z}_i$, which we can rewrite as

$$\sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \mathbf{T} \cdot \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) - \sum_{i \in \text{SS} \cap \text{CS}} L_{\text{SS}, i} \cdot \mathbf{T}_i \cdot \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}).$$

To simulate the preceding expression, it suffices³ to simulate

$$\hat{\mathbf{z}} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \mathbf{T} \cdot \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}). \quad (2)$$

Now, thanks to the entropy in $\{\mathbf{r}_i\}_{i \in \text{SS} \setminus \text{CS}}$, we can show that $\hat{\mathbf{z}}$ only leaks

$$\mathbf{A} \cdot \hat{\mathbf{z}} = H_1(\mu, \text{salt}) - \sum_{i \in \text{SS} \cap \text{CS}} \mathbf{w}_i. \quad (3)$$

Therefore, we can simply simulate $\hat{\mathbf{z}}$ by sampling a random gaussian vector $\tilde{\mathbf{z}}$ (of the appropriate norm) and then programming $H_1(\mu, \text{salt})$ according to (3).

We stress that the above simulation strategy crucially uses the fact that we derive salt from $\{(j, \mathbf{w}_j)\}_{j \in \text{SS}}$ and not $\mathbf{w}$. This ensures that every $(\mu, \text{SS}, \{(j, \mathbf{w}_j)\}_{j \in \text{SS}})$ yields a different salt (via collision-resistance of H_2), which allows us to program a different $H_1(\mu, \text{salt})$ each time.

Adaptive security. Given that our simulation strategy for a static adversary does not use rewinding, it is tempting to conjecture that it also works for an adaptive adversary. We will explain why this is not the case, and how to fix the issue (by changing just the simulation strategy; the protocol stays the same). Recall that an adaptive adversary can choose the set CS based on the protocol execution so far, and upon corrupting party i , learns not just $\mathbf{T}_i$ but also the $\mathbf{r}_i$'s corresponding to previous first-round $\mathbf{w}_i$'s. Our simulator does know $\mathbf{r}_i$ and can provide these values upon corruption; the problem lies in our simulation strategy for $\hat{\mathbf{z}}$ in (2) where CS now corresponds to the current set of corrupted parties. Consider an adversary that first asks for all the partial signatures corresponding to (μ, SS) where $\text{SS} = \{1, \dots, T\}$, and then corrupts parties $1, 2, \dots, T-1$ to learn $\mathbf{T}_1, \dots, \mathbf{T}_{T-1}, \mathbf{r}_1, \dots, \mathbf{r}_{T-1}$. From the partial signatures, it can compute the corresponding $\hat{\mathbf{z}}$. With the current simulation strategy, the norm of $\hat{\mathbf{z}} - (\mathbf{r}_1 + \dots + \mathbf{r}_{T-1})$ will be much larger than it should be (roughly $\sqrt{T} \cdot \|\mathbf{r}_1\|$ instead of $\|\mathbf{r}_1\|$).

To address this issue, we modify our simulation strategy for $\hat{\mathbf{z}}$ as follows:

- sample a random gaussian vector $\tilde{\mathbf{y}}$ (with $\|\tilde{\mathbf{y}}\| \approx \|\mathbf{T}\|$), i.e. we use $\tilde{\mathbf{y}}$ as a proxy for $\mathbf{T} \cdot \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w})$;
- simulate $\hat{\mathbf{z}}$ using $\tilde{\mathbf{y}} + \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i$;
- program $H_1(\mu, \text{salt}) = \mathbf{A}\tilde{\mathbf{y}} + \mathbf{w}$ (satisfying (3) as before).⁴

³ We can compute the preceding expression given $\hat{\mathbf{z}}$ in (2) along with $\{\mathbf{T}_i\}_{i \in \text{CS}}$.

⁴ We mention here an additional subtlety that arises for adaptive corruptions, namely the set CS when we simulate the second-round message may be different from that when we program H_1 . Programming $H_1(\mu, \text{salt}) = \mathbf{A}\tilde{\mathbf{y}} + \mathbf{w}$ has the advantage that it is oblivious to CS , unlike the earlier strategy in (3).

Next, we claim that for any $i^* \in \text{SS} \setminus \text{CS}$, we have:

$$\begin{aligned} & \left(\{(j, \mathbf{w}_j)\}_{j \in \text{SS}}, \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \mathbf{T} \cdot \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}), H_1(\mu, \text{salt}), \{\mathbf{r}_i\}_{i \in \text{SS} \setminus \text{CS}, i \neq i^*} \right) \\ & \approx_s \left(\{(j, \mathbf{w}_j)\}_{j \in \text{SS}}, \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \tilde{\mathbf{y}}, \mathbf{A}\tilde{\mathbf{y}} + \mathbf{w}, \{\mathbf{r}_i\}_{i \in \text{SS} \setminus \text{CS}, i \neq i^*} \right). \end{aligned}$$

To prove this claim, we first replace $H_1(\mu, \text{salt})$ on the LHS with $\mathbf{A}\tilde{\mathbf{y}} + \mathbf{w}$. Then, we use

$$(\mathbf{A}\mathbf{r}_{i^*}, \mathbf{r}_{i^*} + \mathbf{T} \cdot \mathbf{G}^{-1}(\mathbf{A}\tilde{\mathbf{y}})) \approx_s (\mathbf{A}\mathbf{r}_{i^*}, \mathbf{r}_{i^*} + \tilde{\mathbf{y}}), \quad \text{given } \mathbf{A}, \tilde{\mathbf{y}}.$$

This follows from noise flooding over $\Lambda_q^\perp(\mathbf{A})$ along with $\|\mathbf{r}_{i^*}\| \gg \max\{\|\mathbf{T}\|, \|\tilde{\mathbf{y}}\|\}$. To prove the adaptive security of our scheme, we need to prove a more complex statement where the index i^* could be adaptively chosen. A simple solution is to guess a random $i^* \in \text{SS}$ and combine this with a hybrid argument across Q_s signing queries, with a total security loss of $T \cdot Q_s$. In order to achieve better concrete parameters using Renyi divergence, we instead carry out a careful analysis of the conditional distributions, using the tool-kit for discrete gaussians developed in [GMPW20, CATZ24].

Finally, to extract a MSIS solution from a forgery, we proceed as in the vanilla GPV signature: we guess which query to H_1 corresponds to the forgery, and we program its output to be $\mathbf{A}\mathbf{z}_0$ for a random gaussian $\mathbf{z}_0$. If the adversary later outputs a forgery $(\mu^*, \text{salt}^*, \mathbf{z}^*)$ such that $\mathbf{A}\mathbf{z}^* = H_1(\mu^*, \text{salt}^*) = \mathbf{A}\mathbf{z}_0$, then we can output $\mathbf{z}_0 - \mathbf{z}^*$, which with overwhelming probability constitutes a valid MSIS solution w.r.t. $\mathbf{A}$.

Optimizations. We use the truncated trapdoor techniques following [CGM19, YJW23, EEN⁺24] to obtain smaller signature sizes. We sample $\mathbf{A}_0 \leftarrow \mathcal{R}_q^{n \times k}$ along with low-norm matrices $\mathbf{R} \leftarrow \mathcal{R}_q^{k \times \ell}$, $\mathbf{E} \leftarrow \mathcal{R}_q^{n \times \ell}$ and define

$$\begin{aligned} \mathbf{A} &= [\mathbf{I}_n \mid \mathbf{A}_0 \mid \mathbf{A}_0\mathbf{R} + \mathbf{E} + \mathbf{G}_1] \in \mathcal{R}_q^{n \times m}, \\ \mathbf{G}_1 &= \lceil q/p^t \rceil \cdot [\mathbf{I}_n \mid p\mathbf{I}_n \mid \cdots \mid p^{t-1}\mathbf{I}_n] \in \mathcal{R}_q^{n \times tn}, \end{aligned}$$

where $m = k + n + \ell$, $\ell = tn$.⁵ We can define $\mathbf{G} := [\mathbf{I}_n \mid \mathbf{G}_1]$ so that

$$\mathbf{A} \cdot \overbrace{\begin{bmatrix} \mathbf{I}_n & -\mathbf{E} \\ & -\mathbf{R} \\ & & \mathbf{I}_\ell \end{bmatrix}}^{\mathbf{T}} = \mathbf{G}.$$

We can then bound $\max_{\mathbf{u} \in \mathcal{R}_q^n} \|\mathbf{T} \cdot \mathbf{G}^{-1}(\mathbf{u})\|$ by roughly $\frac{q}{p^t} + p \cdot \max\{\|\mathbf{R}\|, \|\mathbf{E}\|\}$, which is minimized when $q \approx p^{t+1} \cdot \max\{\|\mathbf{R}\|, \|\mathbf{E}\|\}$. We defer further details to Section 4.4.

3 Preliminaries

3.1 Notation

For any positive integers $k < n$, $[n]$ denotes $\{1, \dots, n\}$, and $[k..n]$ denotes $\{k, \dots, n\}$. We use κ to denote the security parameter. For a sequence of variables $x_1, \dots, x_\ell$, we use $x_{[i]}$ to denote $(x_1, \dots, x_i)$ and $x_{[i..j]}$ to denote $(x_i, \dots, x_j)$.

For a finite set S , $|S|$ denotes the size of S , and $x \leftarrow S$ denotes sampling an element uniformly from S and assigning it to x . For a distribution $\mathcal{D}$, denote $\text{Supp}(\mathcal{D})$ as the support of $\mathcal{D}$, and $x \leftarrow \mathcal{D}$ denotes sampling x according to $\mathcal{D}$. For any $y \in \text{Supp}(\mathcal{D})$, denote $\mathcal{D}(y) := \Pr_{x \leftarrow \mathcal{D}}[x = y]$, and for $S \subseteq \text{Supp}(\mathcal{D})$, denote $\mathcal{D}(S) := \Pr_{x \leftarrow \mathcal{D}}[x \in S]$.

⁵ Previous lattice-based threshold hash-and-sign use $n = t = \ell = 1$, following [EEN⁺24].

For any set T and any function $F : \text{Supp}(\mathcal{D}) \rightarrow T$, denote $F(\mathcal{D})$ as the distribution of $F(x)$ for x sampled from $\mathcal{D}$. We denote by $\mathcal{B}_n$ the closed Euclidean ball of radius 1 around the origin, that is $\mathcal{B}_n := \{\mathbf{x} \in \mathbb{R}^n : \|\mathbf{x}\| = 1\}$.

For a distribution $\mathcal{D}$, we denote $\mathcal{D}^k$ as a matrix $\mathbf{M} = (\mathbf{m}_1, \dots, \mathbf{m}_k)$ with k columns, and each column is drawn i.i.d. from distribution $\mathcal{D}$, where $\text{Supp}(\mathcal{D})$ is a column vector space.

For a matrix $\mathbf{A} = (\mathbf{a}_1, \dots, \mathbf{a}_n)$ we denote its Gram-Schmidt orthogonalization by $\tilde{\mathbf{A}} = (\tilde{\mathbf{a}}_1, \dots, \tilde{\mathbf{a}}_n)$. And we use $\|\mathbf{A}\|$ to denote the maximum norm among its columns, namely, $\max_{i \in [n]} \|\mathbf{a}_i\|$.

3.2 Polynomial Rings

Let q be an odd prime and d be a power of 2. We denote the ring $\mathcal{R} := \mathbb{Z}[X]/(X^d + 1)$, contained in the cyclotomic field $K := \mathbb{Q}[X]/(X^d + 1)$, and let $\mathcal{R}_q := \mathcal{R}/q\mathcal{R} \cong \mathbb{Z}_q[X]/(X^d + 1)$. Denote $K_{\mathbb{R}} := \mathbb{R} \otimes K \cong \mathbb{R}[X]/(X^d + 1)$. For an element $v \in K_{\mathbb{R}}$, where $v = \sum_{i=0}^{d-1} v_i X^i$, we denote its conjugate as $v^* = \sum_{i=0}^{d-1} -v_i X^{d-i}$. We use ϕ to denote the coefficient embedding that embeds $K_{\mathbb{R}}$ in $\mathbb{R}^d$, and ϕ maps v to vector $(v_0, \dots, v_{d-1}) \in \mathbb{R}^d$. When applying ϕ to a vector $\mathbf{v} \in K_{\mathbb{R}}^m$, ϕ maps $\mathbf{v}$ to a vector in $\mathbb{R}^{md}$ by applying ϕ to each entry of $\mathbf{v}$. The map ϕ is a bijection, and we denote its inverse by ϕ^{-1} . An ℓ_p -norm of $\mathbf{v} \in K_{\mathbb{R}}^m$ is given by

$$\|\mathbf{v}\|_p := \|\phi(\mathbf{v})\|_p = \left(\sum_{i=1}^m \sum_{j=0}^{d-1} |v_{i,j}|^p \right)^{\frac{1}{p}},$$

where $v_{i,j}$ denotes the coefficient of X^j of the i -th entry of $\mathbf{v}$. Additionally, the ℓ_{∞} -norm of $\mathbf{v}$ is defined as $\|\mathbf{v}\|_{\infty} := \max_{i \in [m], j \in [0..d-1]} |v_{i,j}|$. For the ℓ_2 -norm, we omit the subscript and denote $\|\mathbf{v}\|$ as the ℓ_2 -norm of $\mathbf{v}$. Denote the conjugate transpose of $\mathbf{v} \in K_{\mathbb{R}}^m$ as $\mathbf{v}^{\dagger} := (\mathbf{v}^*)^{\top}$. We define the inner product of two vectors $\mathbf{v}, \mathbf{v}' \in K_{\mathbb{R}}^m$ as $\langle \mathbf{v}, \mathbf{v}' \rangle := \phi(\mathbf{v})^{\top} \phi(\mathbf{v}') = \langle \phi(\mathbf{v}), \phi(\mathbf{v}') \rangle$. We have $\|\mathbf{v}\| = \langle \mathbf{v}, \mathbf{v} \rangle$. We say $\mathbf{v}$ is a unit vector if $\|\mathbf{v}\| = 1$.

Also, we define a map ϕ_M that maps each element in $K_{\mathbb{R}}$ to a matrix in $\mathbb{R}^{d \times d}$ as follows. Let $M_X := \begin{pmatrix} \mathbf{0} & -1 \\ \mathbf{I}_{d-1} & \mathbf{0} \end{pmatrix} \in \mathbb{R}^{d \times d}$, where $\mathbf{I}_{d-1}$ is the identity matrix in $\mathbb{R}^{d-1}$. For each $v \in K_{\mathbb{R}}$, $\phi_M(v) := \sum_{i=0}^{d-1} v_i M_X^i$, which can be viewed as the matrix representation of v . In particular, for ϕ and ϕ_M , the following properties hold: for any $v, v' \in K_{\mathbb{R}}$, $\phi_M(v^*) = \phi_M(v)^{\top}$, $\phi_M(vv') = \phi_M(v)\phi_M(v')$ and $\phi_M(v)\phi(v') = \phi(vv')$. We extend the above definitions to $\mathcal{R}_q$ by representing each $v \in \mathcal{R}_q$ as $v = \sum_{i=0}^{d-1} v_i X^i$, where $v_i \in \{-(q-1)/2, \dots, (q-1)/2\}$.

For a matrix $\mathbf{M} \in K_{\mathbb{R}}^{m \times m}$, we denote its conjugate transpose as $\mathbf{M}^{\dagger} = (\mathbf{M}^*)^{\top}$, and we say $\mathbf{M}$ is *hermitian* if $\mathbf{M} = \mathbf{M}^{\dagger}$. We say $\mathbf{M}$ is *positive definite* if and only if $\mathbf{M}$ is hermitian and for all $\mathbf{x} \in K_{\mathbb{R}}^m \setminus \{\mathbf{0}\}$, $\langle \mathbf{x}, \mathbf{M}\mathbf{x} \rangle > 0$, or equivalently, $\phi_M(\mathbf{M})$ is positive definite. Also, denote $\sigma_{\min}(\mathbf{M}) := \inf_{\mathbf{x} \in K_{\mathbb{R}}^m, \|\mathbf{x}\|=1} \langle \mathbf{x}, \mathbf{M}\mathbf{x} \rangle$ as the smallest singular value of $\mathbf{M}$ and $\sigma_{\max}(\mathbf{M}) := \sup_{\mathbf{x} \in K_{\mathbb{R}}^m, \|\mathbf{x}\|=1} \langle \mathbf{x}, \mathbf{M}\mathbf{x} \rangle$ as the largest singular value of $\mathbf{M}$.

3.3 Lattices and Discrete Gaussian Distributions

In this subsection, we give definitions for lattices and discrete Gaussian distributions over $\mathbb{R}$ and $K_{\mathbb{R}}$. An m -dimensional lattice Λ over $\mathbb{Z}$ (resp. $\mathcal{R}$) is a discrete additive subgroup of $\mathbb{Z}$ (resp. $\mathcal{R}$). Equivalently, $\Lambda = \Lambda(\{\mathbf{b}_1, \dots, \mathbf{b}_k\}) := \{\sum_{i \in [k]} x_i \mathbf{b}_i : x_i \in \mathbb{Z}\}$ for a set of linearly independent vectors $\mathbf{b}_1, \dots, \mathbf{b}_k \in \mathbb{Z}^m$ (resp. $\mathcal{R}^m$), which is referred to as a basis of Λ . The size k is the *rank* of the lattice Λ . We say Λ is a *full rank* lattice if $k = m$ (resp. $k = md$ for Λ over $\mathcal{R}$). For any $\mathbf{a} \in \mathbb{Z}^m$ (resp. $\mathcal{R}^m$), $\Lambda + \mathbf{a}$ is a *coset* of Λ . The *dual lattice* of Λ is denoted as $\Lambda^* = \{\mathbf{x} \in \text{Span}(\Lambda) : \forall \mathbf{y} \in \Lambda, \langle \mathbf{x}, \mathbf{y} \rangle \in \mathbb{Z}\}$. A Λ -subspace is the linear span of some subset of Λ , i.e., a subspace S such that $S = \text{Span}(S \cap \Lambda)$.

The *minimum distance* of a n -dimensional lattice Λ , denoted $\lambda_1(\Lambda)$, is the minimum distance between any two distinct lattice points. Equivalently, it is the Euclidean norm of a shortest nonzero lattice vector:

$$\lambda_1(\Lambda) := \min\{\|\mathbf{x} - \mathbf{y}\| : \mathbf{x} \neq \mathbf{y} \in \Lambda\} = \min\{\|\mathbf{x}\| : \mathbf{x} \in \Lambda \setminus \{\mathbf{0}\}\}.$$

More generally, the i -th successive minimum $\lambda_i(\Lambda)$ is the smallest radius r such that the ball $r \cdot \mathcal{B}_n$ contains i linearly independent lattice vectors:

$$\lambda_i(\Lambda) := \min\{r : \dim(\text{span}(\Lambda \cap r \cdot \mathcal{B}_n)) \geq i\}.$$

For a matrix $\mathbf{A} \in \mathcal{R}_q^{k \times m}$, we define the $\mathcal{R}$ -lattice $\Lambda_q^\perp(\mathbf{A}) \subseteq \mathcal{R}^m$ as

$$\Lambda_q^\perp(\mathbf{A}) := \{\mathbf{x} \in \mathcal{R}^m : \mathbf{A}\mathbf{x} = \mathbf{0} \pmod{q}\}.$$

We know $\Lambda_q^\perp(\mathbf{A})$ has full-rank since $q\mathcal{R}^m \subseteq \Lambda_q^\perp(\mathbf{A})$.

For a positive definite matrix $\Sigma \in \mathbb{R}^{m \times m}$ (resp. a positive definite matrix $\Sigma \in K_{\mathbb{R}}^{m \times m}$) and a vector $\mathbf{c} \in \mathbb{R}^m$ (resp. $K_{\mathbb{R}}^m$), we define the function $\rho_{\Sigma, \mathbf{c}}$ over $\mathbb{R}^m$ (resp. $K_{\mathbb{R}}^m$) as

$$\rho_{\Sigma, \mathbf{c}}(\mathbf{x}) := \exp(-\pi \langle \mathbf{x} - \mathbf{c}, \Sigma^{-1}(\mathbf{x} - \mathbf{c}) \rangle).$$

Then, we denote $\mathcal{D}_{\Lambda + \mathbf{a}, \Sigma, \mathbf{c}}$ as the discrete Gaussian distribution over a lattice coset $\Lambda + \mathbf{a} \subseteq \mathbb{Z}^m$ (resp. $\mathcal{R}^m$) with covariance matrix Σ , centered at $\mathbf{c} \in \mathbb{R}^m$, where for $\mathbf{x} \in \Lambda + \mathbf{a}$, we define

$$\mathcal{D}_{\Lambda + \mathbf{a}, \Sigma, \mathbf{c}}(\mathbf{x}) := \Pr[\mathbf{x} \leftarrow \mathcal{D}_{\Lambda + \mathbf{a}, \Sigma, \mathbf{c}}] = \frac{\rho_{\Sigma, \mathbf{c}}(\mathbf{x})}{\rho_{\Sigma, \mathbf{c}}(\Lambda + \mathbf{a})},$$

where $\rho_{\Sigma, \mathbf{c}}(\Lambda + \mathbf{a}) = \sum_{\mathbf{x} \in \Lambda + \mathbf{a}} \rho_{\Sigma, \mathbf{c}}(\mathbf{x})$.

The smoothing parameter of a lattice Λ with respect to $\varepsilon > 0$, denoted by $\eta_\varepsilon(\Lambda)$, is the smallest $s > 0$ such that $\rho_{1/s}(\Lambda^*) \leq 1 + \varepsilon$.

We collect several useful lemmas on discrete Gaussian distributions that will be used throughout the paper.

Lemma 1 ([AGHS13, Lemma 3]). For any $\varepsilon \in (0, 1)$, a lattice $\Lambda \subseteq \mathcal{R}^m$, $\mathbf{c} \in K_{\mathbb{R}}^m$, and $\sigma \geq \eta_\varepsilon(\Lambda)$, then $\Pr[\|\mathbf{x} - \mathbf{c}\| \geq \sigma\sqrt{md} : \mathbf{x} \leftarrow \mathcal{D}_{\Lambda, \sigma, \mathbf{c}}] \leq \frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-md}$.

Lemma 2 ([Pei06, Lemma 2.11]). For any n -dimensional full rank lattice Λ , center $\mathbf{c} \in \mathbb{R}^n$, positive $\varepsilon > 0$, and $s \geq 2\eta_\varepsilon(\Lambda)$, and for every $\mathbf{x} \in \Lambda$, we have

$$D_{\Lambda, s, \mathbf{c}}(\mathbf{x}) \leq \frac{1 + \varepsilon}{1 - \varepsilon} \cdot 2^{-n}.$$

In particular, for $\varepsilon < \frac{1}{3}$, the min-entropy of $D_{\Lambda, s, \mathbf{c}}$ is at least $n - 1$.

Lemma 3 ([GPV08, Theorem 3.1]). For any full-rank lattice $\Lambda(\mathbf{B})$ in $\mathbb{R}^m$ and $\varepsilon > 0$, $\eta_\varepsilon(\Lambda) \leq \frac{\sqrt{\log(2m(1+1/\varepsilon))}}{\pi} \|\tilde{\mathbf{B}}\|$.

Lemma 4 ([GPV08, Corollary 2.8]). Let Λ, Λ' be n -dimensional lattices, with $\Lambda' \subseteq \Lambda$. Then for any $\varepsilon \in (0, \frac{1}{2})$, any $s \geq \eta_\varepsilon(\Lambda')$, and any $\mathbf{c} \in \mathbb{R}^n$, the distribution of $\mathcal{D}_{\Lambda, s, \mathbf{c}} \pmod{\Lambda'}$ is within statistical distance at most 2ε of uniform over $\Lambda \pmod{\Lambda'}$.

We adopt the notation $P \stackrel{\varepsilon}{\approx} Q$ from [GMPW20]: for any two distributions P, Q with the same support and $\varepsilon > 0$, we say that $P \stackrel{\varepsilon}{\approx} Q$ if and only if $\max_{x \in \text{Supp}(P)} |\log P(x) - \log Q(x)| \leq \log(1 + \varepsilon)$, or equivalently, $\max(R_\infty(P\|Q), R_\infty(Q\|P)) \leq 1 + \varepsilon$. Note that if $P \stackrel{\varepsilon}{\approx} Q$, then the statistical distance between P and Q is bounded by $\varepsilon/2$, i.e., $\frac{1}{2} \sum_{x \in \text{Supp}(P)} |P(x) - Q(x)| \leq \frac{\varepsilon}{2}$.

Lemma 5 (Adapted from [GMPW20, Lemma 2.6]). For any $\varepsilon \in (0, 1)$ defining $\varepsilon' = 2\varepsilon/(1 - \varepsilon)$, $\sigma > 0$, lattice coset $\Lambda + \mathbf{a} \subseteq K_{\mathbb{R}}^m$, and matrix $\mathbf{T} \in K^{k \times m}$ such that $\phi_{\mathbf{M}}(\mathbf{T})$ has full-row rank, $\ker(\mathbf{T})$ is a Λ -subspace and $\eta_\varepsilon(\Lambda \cap \ker(\mathbf{T})) \leq \sigma$, we have

$$\mathbf{T} \cdot \mathcal{D}_{\Lambda + \mathbf{a}, \sigma} \stackrel{\varepsilon'}{\approx} \mathcal{D}_{\mathbf{T}\Lambda + \mathbf{T}\mathbf{a}, \sigma^2 \mathbf{T}\mathbf{T}^\dagger}.$$

Lemma 6 (Adapted from [CATZ24, Lemma 11]). For any constant $\varepsilon \in (0, 1)$, $\sigma_r > 0$, $1 \leq k \leq t - 1$, full rank $\mathcal{R}$ -lattice $\Lambda \subseteq \mathcal{R}^m$ with $2\eta_\varepsilon(\Lambda) \leq \sigma_r$, arbitrary elements $\mathbf{s}_1, \dots, \mathbf{s}_t \in \mathcal{R}^m$. Let $\mathbf{r}_1, \dots, \mathbf{r}_t$ be independent samples with

$$\mathbf{r}_i \leftarrow \mathcal{D}_{\Lambda + \mathbf{s}_i, \sigma_r} \text{ and } \mathbf{P} = \begin{pmatrix} \mathbf{I}_k & \mathbf{0} \\ 1, 1, \dots, 1 \end{pmatrix} \in \mathcal{R}^{(k+1) \times t} \text{ and } \begin{pmatrix} \mathbf{y}_1 \\ \mathbf{x} \end{pmatrix} := \begin{pmatrix} \mathbf{y}_1 \\ \vdots \\ \mathbf{y}_{k+1} \end{pmatrix} = (\mathbf{P} \otimes \mathbf{I}_m) \begin{pmatrix} \mathbf{r}_1 \\ \vdots \\ \mathbf{r}_t \end{pmatrix}.$$

Denote the joint distribution of $\begin{pmatrix} \mathbf{y}_1 \\ \mathbf{x} \end{pmatrix}$ as $\mathcal{D}$. Then,

$$\mathcal{D} \stackrel{\varepsilon''}{\approx} \mathcal{D}_{(\mathbf{P} \otimes \mathbf{I}_m)\Lambda^t + \mathbf{c}, \Sigma \otimes \mathbf{I}_m},$$

Game MSIS$_{q,d,n,m,\beta_{\text{sis}}}^{\mathcal{A}}$: $\mathbf{A} \leftarrow \mathcal{R}_q^{n \times m}$ $\mathbf{x} \leftarrow \mathcal{A}(\mathbf{A})$ Return ($\ \mathbf{x}\ \leq \beta_{\text{sis}} \wedge \mathbf{A}\mathbf{x} = \mathbf{0}$) Game HNF-MSIS$_{q,d,n,m,\beta_{\text{sis}}}^{\mathcal{A}}$: $\mathbf{A}' \leftarrow \mathcal{R}_q^{n \times (m-n)}, \mathbf{A} := [\mathbf{I}_n \mid \mathbf{A}']$ $\mathbf{x} \leftarrow \mathcal{A}(\mathbf{A})$ Return ($\ \mathbf{x}\ \leq \beta_{\text{sis}} \wedge \mathbf{A}\mathbf{x} = \mathbf{0}$)	Game MLWE$_{q,d,n,m,\beta_{\text{lwe}}}^{\mathcal{A}}$: $\mathbf{A}' \leftarrow \mathcal{R}_q^{n \times (m-n)}, \mathbf{A} := [\mathbf{I}_n \mid \mathbf{A}']$ $\mathbf{s} \leftarrow \mathcal{B}_{\beta_{\text{lwe}}}^m; \mathbf{t}_0 \leftarrow \mathbf{A}\mathbf{s}; \mathbf{t}_1 \leftarrow \mathcal{R}_q^n$ $b \leftarrow \{0, 1\}$ $\bar{b} \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{t}_b)$ Return ($\bar{b} = b$) Game HNF-MSIS'$_{q,d,n,k,\ell,\beta_{\text{sis}},\beta_{\text{lwe}}}^{\mathcal{A}}$: $\mathbf{A}_0 \leftarrow \mathcal{R}_q^{n \times k}, \mathbf{R} \leftarrow \mathcal{B}_{\beta_{\text{lwe}}}^{k \times \ell}, \mathbf{E} \leftarrow \mathcal{B}_{\beta_{\text{lwe}}}^{n \times \ell}$ $\mathbf{A} := [\mathbf{I}_n \mid \mathbf{A}_0 \mid \mathbf{A}_0\mathbf{R} + \mathbf{E}] \in \mathcal{R}_q^{n \times m}$, $m = k + n + \ell$ $\mathbf{x} \leftarrow \mathcal{A}(\mathbf{A})$ Return ($\ \mathbf{x}\ \leq \beta_{\text{sis}} \wedge \mathbf{A}\mathbf{x} = \mathbf{0}$)
---	--

Fig. 2. The module (HNF-)SIS, LWE, HNF-SIS' problem, where $\mathcal{R} := \mathbb{Z}[X]/(X^d + 1)$, $\mathcal{R}_q := \mathcal{R}/q\mathcal{R}$ and $\mathcal{B}_\beta := \{x \in \mathcal{R} : \|x\|_\infty \leq \beta\}$.

where $\varepsilon'' = \frac{2((1+\varepsilon)^{t-k-1}-1)}{2-(1+\varepsilon)^{t-k-1}}$, Λ^t is the t -th direct sum of Λ , which is a tm -dimensional lattice over $\mathcal{R}$, $\Sigma = \sigma_r^2 \mathbf{P}\mathbf{P}^\dagger \in$

$K_{\mathbb{R}}^{(k+1) \times (k+1)}$ is positive definite, and $\mathbf{c} = (\mathbf{P} \otimes \mathbf{I}_m) \cdot \begin{pmatrix} \mathbf{s}_1 \\ \vdots \\ \mathbf{s}_t \end{pmatrix} \in \mathcal{R}^{(k+1)m}$.

Moreover, denote $\mathcal{D}_1$ as the marginal distribution of $\mathbf{x}$ and $\mathcal{D}_{2|\mathbf{x}_0}$ as the distribution of $\mathbf{y}_1$ conditioning on $\mathbf{x} = \mathbf{x}_0 := (\mathbf{x}_0[1], \dots, \mathbf{x}_0[k])^\top$, for any $\mathbf{x}_0 \in \Lambda^k$, and we have

$$\mathcal{D}_1 \stackrel{\varepsilon''}{\approx} \mathcal{D}_{\Lambda^k + \mathbf{a}_1, \sigma_r^2 \Sigma_{22}}, \quad \mathcal{D}_{2|\mathbf{x}_0} \stackrel{\varepsilon''}{\approx} \mathcal{D}_{\Lambda + \mathbf{s}_1, V_{11}^{-1} \cdot \mathbf{I}_m, -V_{11}^{-1} \cdot (\mathbf{V}_{21} \otimes \mathbf{I}_m) \cdot \mathbf{x}_0},$$

where $\Sigma = \begin{pmatrix} \sigma_r^2 & \sigma_r^2 \mathbf{e}_k^\top \\ \sigma_r^2 \mathbf{e}_k & \sigma_r^2 \Sigma_{22} \end{pmatrix}$, $\Sigma^{-1} := \begin{pmatrix} V_{11} & \mathbf{V}_{21} \\ \mathbf{V}_{12} & \mathbf{V}_{22} \end{pmatrix}$, $\mathbf{a}_1 = \begin{pmatrix} \mathbf{s}_2 \\ \vdots \\ \mathbf{s}_k \\ \sum_{i \in [t]} \mathbf{s}_i \end{pmatrix} \in \mathcal{R}^{km}$, where $\mathbf{P}_1 = \begin{pmatrix} \mathbf{I}_{k-1} & \mathbf{0} \\ 1, 1, \dots, 1 \end{pmatrix} \in \mathcal{R}^{k \times t}$,
 $V_{11} = \frac{1}{\sigma_r^2} \cdot (1 + \frac{1}{t-k})$, $\mathbf{V}_{21} = \frac{1}{\sigma_r^2} \cdot (\frac{1}{t-k}, \dots, \frac{1}{t-k}, -\frac{1}{t-k})$.

The proof of Lemma 6 is deferred to Appendix A.

3.4 Lattice Assumptions

We recall the module short integer solution (MSIS) problem, its Hermite normal form variants, and the module learning with errors (MLWE) problem, as defined in Fig. 2. We also introduce a variant of MSIS, denoted the HNF-MSIS' problem. Recall that MSIS can be reduced to its Hermite normal form variants [Pei15]. In addition, we show that the HNF-MSIS' problem can be reduced to the HNF-MSIS problem together with the MLWE problem.

The advantage of $\mathcal{A}$ for the (HNF-)MSIS problem is defined as

$$\text{Adv}_{q,d,n,m,\beta_{\text{sis}}}^{(\text{hnf-})\text{msis}}(\mathcal{A}) := \Pr [(\text{HNF-})\text{MSIS}_{q,d,n,m,\beta_{\text{sis}}}^{\mathcal{A}} = 1].$$

The advantage of $\mathcal{A}$ for the MLWE problem are defined as

$$\text{Adv}_{q,d,n,m,\beta_{\text{lwe}}}^{\text{mlwe}}(\mathcal{A}) := |\Pr [\text{MLWE}_{q,d,n,m,\beta_{\text{lwe}}}^{\mathcal{A}} = 1] - \frac{1}{2}|.$$

The advantage of $\mathcal{A}$ for the HNF-MSIS' problem are defined as

$$\text{Adv}_{q,d,n,k,\ell,\beta_{\text{sis}},\beta_{\text{lwe}}}^{\text{hnf-msis}'}(\mathcal{A}) := \Pr [\text{HNF-MSIS}'_{q,d,n,k,\ell,\beta_{\text{sis}},\beta_{\text{lwe}}}^{\mathcal{A}} = 1] .$$

Lemma 7 (Hardness of HNF-MSIS'). *For any adversary $\mathcal{A}$ against the HNF-MSIS' game, there exist an MSIS adversary $\mathcal{B}$ and an MLWE adversary $\mathcal{C}$ running in time roughly that of $\mathcal{A}$, such that*

$$\text{Adv}_{q,d,n,k,\ell,\beta_{\text{sis}},\beta_{\text{lwe}}}^{\text{hnf-msis}'}(\mathcal{A}) \leq \text{Adv}_{q,d,n,m,\beta_{\text{sis}}}^{\text{hnf-msis}}(\mathcal{B}) + \ell \cdot \text{Adv}_{q,d,n,n+k,\beta_{\text{lwe}}}^{\text{mlwe}}(\mathcal{C}).$$

Proof. Given an HNF-MSIS' instance of the form $\mathbf{A} = [\mathbf{I}_n \mid \mathbf{A}_0 \mid \mathbf{A}_0\mathbf{R} + \mathbf{E}]$, we can replace the term $\mathbf{A}_0\mathbf{R} + \mathbf{E}$ with a uniformly random matrix $\mathbf{A}' \in \mathcal{R}_q^{n \times \ell}$. The resulting matrix $\mathbf{A} = [\mathbf{I}_n \mid \mathbf{A}_0 \mid \mathbf{A}']$ is then a valid instance of HNF-MSIS. Consequently, we obtain

$$|\text{Adv}_{q,d,n,k,\ell,\beta_{\text{sis}},\beta_{\text{lwe}}}^{\text{hnf-msis}'}(\mathcal{A}) - \text{Adv}_{q,d,n,m,\beta_{\text{sis}}}^{\text{hnf-msis}}(\mathcal{B})| \leq \ell \cdot \text{Adv}_{q,d,n,n+k,\beta_{\text{lwe}}}^{\text{mlwe}}(\mathcal{C}).$$

This yields the desired result. $\square$

3.5 Rényi divergence

We define the notion of Rényi divergence between two distributions P, Q which we will use in the analysis of the scheme. For a discrete distribution P , we denote the support of P as $\text{Supp}(P) := \{x : P(x) > 0\}$.

Definition 1 (Rényi Divergence). *Let P, Q be two discrete probability distributions such that $\text{Supp}(P) \subseteq \text{Supp}(Q)$ and $\alpha \in [1, +\infty]$. We define the Rényi Divergence of order α , for $\alpha \in (1, \infty)$, as*

$$R_\alpha(P\|Q) := \left(\sum_{x \in \text{Supp}(P)} \frac{P(x)^\alpha}{Q(x)^{\alpha-1}} \right)^{\frac{1}{\alpha-1}} .$$

For $\alpha = 1$ and $\alpha = \infty$, we define

$$R_1(P\|Q) := \exp \left(\sum_{x \in \text{Supp}(P)} P(x) \log \frac{P(x)}{Q(x)} \right), \quad R_\infty(P\|Q) := \max_{x \in \text{Supp}(P)} \frac{P(x)}{Q(x)} .$$

Below, we collect several useful lemmas on Rényi divergence that will be used in the analysis of the scheme.

Lemma 8 ([ASY22, Lemma 2.27]). *Let $\alpha \in [1, \infty]$ and P, Q be discrete probability distributions with $\text{Supp}(P) \subseteq \text{Supp}(Q)$. Then, the following properties hold:*

- **Log Positivity:** $R_\alpha(P\|Q) \geq R_\alpha(P\|P) = 1$.
- **Data Processing Inequality:** $R_\alpha(P^f\|Q^f) \leq R_\alpha(P\|Q)$ for any function f , where P^f (and Q^f) denotes the distribution which samples $x \leftarrow P$ ($x \leftarrow Q$) and outputs $f(x)$.
- **Probability Preservation:** Let $E \subseteq \text{Supp}(Q)$ be an arbitrary event. Then, for $\alpha \in (1, \infty)$,

$$\Pr_{x \leftarrow P}[E]^{\alpha/(\alpha-1)} / R_\alpha(P\|Q) \leq \Pr_{x \leftarrow Q}[E].$$

- **Weak Triangle Inequality:** Let P_1, P_2, P_3 be three probability distributions where $\text{Supp}(P_1) \subseteq \text{Supp}(P_2) \subseteq \text{Supp}(P_3)$. Then, we have

$$R_\alpha(P_1\|P_3) \leq \begin{cases} R_\alpha(P_1\|P_2) \cdot R_\alpha(P_2\|P_3) \\ R_\alpha(P_1\|P_2)^{\frac{\alpha}{\alpha-1}} \cdot R_\alpha(P_2\|P_3) & \text{if } \alpha \in (1, \infty) \end{cases} .$$

Lemma 9 ([Ros20, Proposition 2]). Let P and Q denote two distributions of a sequence of random variables $(X_1, \dots, X_n)$. For $1 \leq i \leq n$, denote $P_{i|x_{[i-1]}}$ (resp. $Q_{i|x_{[i-1]}}$) as the conditional distribution of X_i given $(X_1, \dots, X_{i-1}) = (x_1, \dots, x_{i-1})$. Then, for any $\alpha > 1$,

$$R_\alpha(P\|Q) \leq \prod_{i \in [n]} \max_{x_{[i-1]}} R_\alpha(P_{i|x_{[i-1]}}\|Q_{i|x_{[i-1]}}).$$

Lemma 10 ([TT15, Lemma 5]). For any m -dimensional lattice $\Lambda \subseteq \mathcal{R}^m$, $\sigma \in \mathbb{R}$ and $\mathbf{c}, \mathbf{c}' \in \Lambda$, then

$$R_\alpha(\mathcal{D}_{\Lambda, \sigma, \mathbf{c}}\|\mathcal{D}_{\Lambda, \sigma, \mathbf{c}'}) \leq \exp\left(\frac{\alpha\pi\|\mathbf{c} - \mathbf{c}'\|^2}{\sigma^2}\right)$$

If $\mathbf{c}, \mathbf{c}' \in K_{\mathbb{R}}^m$, $\sigma \geq \eta_\varepsilon(\Lambda)$, then

$$R_\alpha(\mathcal{D}_{\Lambda, \sigma, \mathbf{c}}\|\mathcal{D}_{\Lambda, \sigma, \mathbf{c}'}) \leq \left(\frac{1+\varepsilon}{1-\varepsilon}\right)^{\frac{\alpha}{\alpha-1}} \exp\left(\frac{\alpha\pi\|\mathbf{c} - \mathbf{c}'\|^2}{\sigma^2}\right)$$

4 Our construction

We first present the formal definitions of partially non-interactive threshold signatures and the security notions, then describe our construction, and finally analyze its security.

4.1 Syntax and security definitions

Syntax. T -out-of- N partially non-interactive threshold signature is a tuple of algorithms $\text{TS} = (\text{Setup}, \text{Sign}_1, \text{Sign}_2, \text{Combine}, \text{Ver})$ that act as follows.

- $\text{Setup}(1^\kappa) \rightarrow (\text{pk}, (\text{st}_i)_{i \in [N]})$: The setup algorithm takes the security parameter as input and outputs a public key pk and an initial state st_i for each signer i .
- $\text{Sign}_1(i, \text{st}_i) \rightarrow (pm_i, \text{st}_i)$: The first-round signing algorithm takes a signer index $i \in [N]$ and the corresponding state of signer i as input and outputs an offline nonce pm_i and the updated state.
- $\text{Sign}_2(i, \text{st}_i, \mu, \text{SS}, \mathcal{M} = \{(j, pm_j)\}_{j \in \text{SS}}) \rightarrow (psig_i, \text{st}_i)$: The second-round signing algorithm takes a signer index i , the corresponding state st_i , a message μ to be signed, a set of signers SS participated, and a set of first-round nonces $\mathcal{M} = \{(j, pm_j)\}_{j \in \text{SS}}$ as input and outputs a partial signature $psig_i$ and the updated state st_i . The algorithm may output $\perp$ if the inputs are invalid.
- $\text{Combine}(\mu, \text{SS}, \mathcal{M}, \{(i, psig_i)\}_{i \in \text{SS}}) \rightarrow sig$: The algorithm takes the second-round input $(\mu, \text{SS}, \mathcal{M})$ and the set of partial signatures $\{(i, psig_i)\}_{i \in \text{SS}}$ as input and outputs the final signature.
- $\text{Ver}(\text{pk}, \mu, sig) \rightarrow 1/0$: The verification algorithm takes the public key pk , a message μ , and a signature sig as input and outputs 1 (resp. 0) if the signature is valid (resp. invalid).

The honest signing process can be described by the correctness game TS-COR defined in Figure 3. In the first (offline) round, each signer i runs the Sign_1 algorithm to generate a first-round nonce pm_i and sends it to a coordinator. The coordinator can be one of the signers. In the second (online) round, to sign a message μ with the set of involved signers SS , the coordinator sends $(\mu, \text{SS}, \mathcal{M} = \{(j, pm_j)\}_{j \in \text{SS}})$ to each signer $i \in \text{SS}$. Each signer i runs the Sign_2 algorithm and sends the partial signature $psig_i$ to the coordinator. Finally, the coordinator runs Combine to obtain the final signature. We say that the scheme TS is correct with correctness error ε_{cor} if for any $\mu \in \{0, 1\}^*$ and $\text{SS} \subseteq [N]$ such that $|\text{SS}| \geq T$ and $\Pr[\text{TS-COR}_{\text{TS}}(\kappa, \mu, \text{SS}) = 1] \geq 1 - \varepsilon_{\text{cor}}$.

Unforgeability. Unforgeability guarantees that the adversary cannot forge a signature for a message that is not considered signed even if the adversary can corrupt the coordinator and fewer than T signers. As pointed out by Bellare et al. [BTZ22], there are different ways to define whether a message is considered signed, which leads to different security levels. Here, we adopt the strongest security notion, TS-UF-4 from [BTZ22] and extended it to the adaptive setting, refer to as adp-TS-UF-4 . A similar definition is also used in [ZT25].

```

Game TS-CORTS( $\kappa, \mu, SS$ ) :
  ( $pk, (st_i)_{i \in [N]}$ )  $\leftarrow$  Setup( $1^\kappa$ )
  For  $i \in SS$  do
    ( $pm_i, st_i$ )  $\leftarrow$  Sign1( $i, st_i$ )
  For  $i \in SS$  do
    ( $psig_i, st_i$ )  $\leftarrow$  Sign2( $i, st_i, \mu, SS, \{(j, pm_j)\}_{j \in SS}$ )
   $sig \leftarrow$  Combine( $\mu, SS, \mathcal{M}, \{(i, psig_i)\}_{i \in SS}$ )
  Return Ver( $pk, \mu, sig$ )

```

Fig. 3. The TS-COR game for a threshold signature scheme TS for N signers and threshold T .

<pre> Game adp-TS-UF-4_{TS}^A(κ), adp-TS-sUF-4_{TS}^A(κ) : ($pk, (st_i)_{i \in [N]}$) $\leftarrow$ Setup(1^κ); $H \leftarrow$ TS.HF $CS = \emptyset$; TrivQ = $\emptyset$; curSS = $()$ (μ^*, sig^*) $\leftarrow$ $\mathcal{A}^{\text{INIT}, \text{OSIGN}_1, \text{OSIGN}_2, \text{COR}, \text{RO}}$($pk$) If $\exists (\mu, SS, \mathcal{M}) \in \text{TrivQ} : \mu = \mu^*$ $\wedge \text{SVer}(pk, \mu, SS, \mathcal{M}, sig^*) = 1$ then Return 0 Return Ver(pk, μ^*, sig^*) Oracle COR(i) : Require: $i \in [N] \setminus CS$ and $CS < T - 1$ $CS = CS \cup \{i\}$ Return st_i Oracle RO(x) : Return $H(x)$ </pre>	<pre> Oracle OSIGN₁(i) : Require: $i \in [N] \setminus CS$ (pm_i, st_i) $\leftarrow$ Sign₁(i, st_i) Return pm_i Oracle OSIGN₂($i, \mu, SS, \mathcal{M}$) : Require: $SS \subseteq [N]$ and $SS \geq T$ ($psig_i, st_i$) $\leftarrow$ Sign₂($i, st_i, \mu, SS, \mathcal{M}$) If $psig_i = \perp$ then return $\perp$ If curSS($\mu, SS, \mathcal{M}$) = $\perp$ then curSS($\mu, SS, \mathcal{M}$) = $\emptyset$ curSS($\mu, SS, \mathcal{M}$) = curSS($\mu, SS, \mathcal{M}$) $\cup \{i\}$ If $SS \subseteq CS \cup \text{curSS}(\mu, SS, \mathcal{M})$ then TrivQ $\leftarrow$ TrivQ $\cup \{\mu, SS, \mathcal{M}\}$ Return $psig_i$ </pre>
--	--

Fig. 4. The adaptive unforgeability games adp-TS-UF-0 and adp-TS-sUF-4 for a threshold signature scheme TS for N signers and threshold T , where adp-TS-UF-4 contains all but the highlight box and adp-TS-sUF-4 contains everything. The games are defined in the random oracle model, where the hash function used by the algorithms of TS is modeled as a random oracle, and TS.HF denotes the space of the hash functions from which the random oracle is drawn.

The adp-TS-UF-4 game is defined in Figure 4. In adp-TS-UF-4, a message μ is considered signed if and only if the adversary made a second-round signing query $(\mu, SS, \mathcal{M})$ for some $SS \subseteq [N]$ and $\mathcal{M}$ such that all honest signers in SS responded to the query, i.e., each honest signer received the query and the Sign₂ algorithm did not return $\perp$.

The game is defined in the random oracle model, where the hash function used by the algorithms of TS is modeled as a random oracle. The advantage of an adversary $\mathcal{A}$ playing the adp-TS-UF-4 game is defined as $\text{Adv}_{\text{TS}}^{\text{adp-ts-uf-4}}(\mathcal{A}, \kappa) := \Pr[\text{adp-TS-UF-4}_{\text{TS}}^{\mathcal{A}}(\kappa) = 1]$.

Strong unforgeability. Further, we consider strong unforgeability, first proposed by Bellare et al. [BTZ22] in the setting of threshold signatures. Informally, this notion guarantees that an adversary cannot produce a signature that is not considered issued, analogous to strong unforgeability for standard signature schemes.

To define strong unforgeability of a scheme TS, we require an additional algorithm SVer defined in TS that takes a second round query $(\mu, SS, \mathcal{M})$ and a signature sig as input and checks whether the signature is *associated* with the query. The strong unforgeability game adp-TS-sUF-4 is defined in Figure 4. In adp-TS-sUF-4, a signature sig is considered issued for a message μ if and only if the adversary made a second-round signing query $(\mu, SS, \mathcal{M})$ for some $SS \subseteq [N]$ and $\mathcal{M}$ such that (1) sig is valid and associated with the query and (2) all honest signers in SS responded to

Setup(1^κ) : $(\mathbf{A}, \mathbf{T}) \leftarrow \text{KeyGen}(1^\kappa)$ satisfying $\mathbf{AT} = \mathbf{G}$, where $\mathbf{A} \in \mathcal{R}_q^{n \times m}$, $\mathbf{T} \in \mathcal{R}_q^{m \times \ell}$, $\mathbf{G} \in \mathcal{R}_q^{n \times \ell}$ For $(i, j) \in [N] \times [N]$ do $\text{seed}_{i,j} \leftarrow \{0, 1\}^{2\kappa}$ $\tilde{\mathbf{T}}_1, \dots, \tilde{\mathbf{T}}_{T-1} \leftarrow \mathcal{R}_q^{m \times \ell}$ For $i \in [N]$ do $\mathbf{T}_i = \mathbf{T} + \sum_{j=1}^{T-1} \tilde{\mathbf{T}}_j i^j$ $\text{sk}_i = (\mathbf{T}_i, (\text{seed}_{i,j}, \text{seed}_{j,i})_{j \in [N]})$ $\text{st}_i.\text{sk} = \text{sk}_i$ $\text{st}_i.\text{mapPP} = ()$; $\text{st}_i.\text{validPP} = ()$ Return $\text{pk} = \mathbf{A}, (\text{st}_i)_{i \in [N]}$. Sign₁($i, \text{st}_i$) : Require: $i \in [N]$ $\mathbf{r}_i \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_r}$; $\mathbf{w}_i = \mathbf{A}\mathbf{r}_i$ $\text{st}_i.\text{mapPP}(\mathbf{w}_i) = \mathbf{r}_i$ $\text{st}_i.\text{validPP}(\mathbf{w}_i) = 1$ Return $\mathbf{w}_i, \text{st}_i$ Ver($\text{pk}, \mu, (\mathbf{z}, \text{salt})$) : Return $\ \mathbf{z}\ \leq L_1 \wedge (\mathbf{A}\mathbf{z} = H_1(\mu, \text{salt}))$	Sign₂($i, \text{st}_i, \mu, \text{SS}, \mathcal{M}$) : Require: $i \in \text{SS} \subseteq [N]$ and $\text{SS} \geq T$ $(\mathbf{T}_i, (\text{seed}_{i,j}, \text{seed}_{j,i})_{j \in [N]}) = \text{st}_i.\text{sk}$ $\{(j, \mathbf{w}_j)\}_{j \in \text{SS}} = \mathcal{M}$ If $\text{st}_i.\text{validPP}(\mathbf{w}_i) \neq 1$ then return $\perp$ $\text{st}_i.\text{validPP}(\mathbf{w}_i) = 0$ $\mathbf{r}_i = \text{st}_i.\text{mapPP}(\mathbf{w}_i)$ $\mathbf{w} = \sum_{i \in \text{SS}} \mathbf{w}_i$ $\text{salt} = H_2(\mu, \text{SS}, \mathcal{M})$ $\text{mask}_i \leftarrow \sum_{j \in \text{SS}} H_3(\text{seed}_{i,j}, \text{salt})$ $\text{mask}'_i \leftarrow \sum_{j \in \text{SS}} H_3(\text{seed}_{j,i}, \text{salt})$ $\mathbf{z}_i = \mathbf{r}_i + L_{\text{SS},i} \cdot \mathbf{T}_i \mathbf{G}^{-1} (H_1(\mu, \text{salt}) - \mathbf{w})$ $+ \text{mask}_i - \text{mask}'_i$ Return $\mathbf{z}_i, \text{st}_i$ Combine($\mu, \text{SS}, \mathcal{M}, \{(\mathbf{z}_i, \text{st}_i)\}_{i \in \text{SS}}$) : $\mathbf{z} = \sum_{i \in \text{SS}} \mathbf{z}_i, \text{salt} = H_2(\mu, \text{SS}, \mathcal{M})$ Return $(\mathbf{z}, \text{salt})$ SVer($\text{pk}, \mu, \text{SS}, \mathcal{M}, (\mathbf{z}, \text{salt})$) : Return $\text{salt} = H_2(\mu, \text{SS}, \mathcal{M})$
--	--

Fig. 5. Lattice-based T -out-of- N threshold signatures Oriole. The instantiation of the algorithm KeyGen is provided in Section 4.4. Here $H_1 : \{0, 1\}^* \rightarrow \mathcal{R}_q^n$, $H_2 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ and $H_3 : \{0, 1\}^* \rightarrow \mathcal{R}_q^m$. Also, $\mathbf{G}$ denotes the gadget matrix, and $\mathbf{G}^{-1}$ refers to the deterministic inversion, both defined in Section 4.4. $L_{\text{SS},i}$ denotes the Lagrange coefficient. The public key pk is implicitly given to all algorithms except KeyGen.

the query. In addition, we require that the adversary cannot forge two different valid signatures that are associated with the same second-round query. Note that in [BTZ22], the requirement on SVer is that for each second-round query, there exists at most one signature that is accepted by SVer. Here, we relax this requirement by only requiring that it is computationally hard to find two distinct signatures for the same second-round query. The advantage of an adversary $\mathcal{A}$ playing the adp-TS-sUF-4 game is defined as $\text{Adv}_{\text{TS}}^{\text{adp-ts-suf-4}}(\mathcal{A}, \kappa) := \Pr[\text{adp-TS-sUF-4}_{\text{TS}}^{\mathcal{A}}(\kappa) = 1]$.

4.2 Construction

We present our threshold signature scheme in Figure 5, and refer to Section 2 for a high-level overview. Relative to the scheme described there, we introduce a signer-specific state st_i to facilitate recording previously used $\mathbf{w}_i$. The mask term $\text{mask}_i - \text{mask}'_i$ in Figure 5 corresponds to $\text{mask}_{i, \text{SS}, \text{salt}}$ in the overview, and the partial signature $\mathbf{z}_i$ in Figure 5 corresponds to $\mathbf{z}'_i$ in the overview. The instantiation of the algorithm KeyGen is provided in Section 4.4. The Lagrange coefficient $L_{U,j} \in \mathbb{Z}_q$ for any subset $U \subseteq [n]$ and $j \in U$ is defined as $L_{U,j} := \prod_{i \in U \setminus \{j\}} \frac{-i}{j-i}$. Also, we introduce an additional algorithm SVer, which is used to define strong unforgeability and will be discussed further in Section 4.3.

The following theorem establishes the correctness of our scheme.

Theorem 1 (correctness). *If $\sigma_r \geq \max\{a_2, 2\eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))\}$, here $a_2 = \max_{\mathbf{u} \in \mathcal{R}_q^n} \|\mathbf{T}\mathbf{G}^{-1}(\mathbf{u})\|$ and $L_1 \geq (\sqrt{mdN} + 1)\sigma_r$, then the Threshold Signature Scheme Oriole in Fig 5 is correct with a correct error $\varepsilon_{\text{cor}} \leq Q_s \cdot \frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-md} + Q_s \cdot \frac{2\varepsilon}{1-\varepsilon} + N \cdot Q_s^2 \cdot (q^{-nd} + 2\varepsilon)$.*

Proof. For every $i \in \text{SS}$, by Lemma 4, the outputs of $\text{Sign}_1(i, \text{st}_i)$, namely $\{\mathbf{w}_i^j\}_{j \in [Q_s]}$, are distinct except with probability at most $Q_s^2 \cdot (q^{-nd} + 2\varepsilon)$, where $\mathbf{w}_i^j = \mathbf{A}\mathbf{r}_i^j$, $\mathbf{r}_i^j \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_r}$ and $\sigma_r \geq 2\eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))$. Consequently, for

any honestly generated $\mathcal{M} = \{(i, \mathbf{w}_i)\}_{i \in \text{SS}}$, there exists $i \in \text{SS}$ such that $\text{Sign}_2(i, \text{st}_i, \mu, \text{SS}, \mathcal{M})$ outputs $\perp$ only with probability at most $N \cdot Q_s^2 \cdot (q^{-nd} + 2\varepsilon)$.

From Figure 5,

$$\begin{aligned} \mathbf{z} &= \sum_{i \in \text{SS}} \mathbf{r}_i + L_{\text{SS},i} \cdot \mathbf{T}_i \cdot \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) + \text{mask}_i - \text{mask}'_i \\ &= \sum_{i \in \text{SS}} \mathbf{r}_i + \mathbf{T} \cdot \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}), \end{aligned}$$

where the second equality uses

$$\mathbf{T} = \sum_{i \in \text{SS}} L_{\text{SS},i} \cdot \mathbf{T}_i, \quad \sum_{i \in \text{SS}} \text{mask}_i = \sum_{i \in \text{SS}} \text{mask}'_i.$$

Therefore,

$$\begin{aligned} \mathbf{A}\mathbf{z} &= \sum_{i \in \text{SS}} \overbrace{\mathbf{A}\mathbf{r}_i}^{=\mathbf{w}_i} + \overbrace{\mathbf{A}\mathbf{T}}^{=\mathbf{G}} \cdot \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) \\ &= \mathbf{w} + (H_1(\mu, \text{salt}) - \mathbf{w}) = H_1(\mu, \text{salt}). \end{aligned}$$

We proceed to bound $\|\mathbf{z}\|$. By Lemma 5, we have $\sum_{i \in \text{SS}} \mathbf{r}_i \approx^{\varepsilon'} \mathcal{D}_{\Lambda_q^\perp(\mathbf{A}) + \mathbf{s}, \sqrt{|\text{SS}|} \cdot \sigma_r}$, where $\mathbf{s}$ is any vector satisfies $\mathbf{A}\mathbf{s} = \mathbf{w}$, $\varepsilon' = 2\varepsilon(1 - \varepsilon)$ and we used $\sigma_r \geq 2\eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))$. By Lemma 1, we obtain $\|\sum_{i \in \text{SS}} \mathbf{r}_i\| \leq \sqrt{mdN}\sigma_r$ except with probability at most $\frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-md} + \varepsilon'$, where we used the fact that $|\text{SS}| \leq N$. By the parameters setting, we have $\|\mathbf{z}\| \leq \|\sum_{i \in \text{SS}} \mathbf{r}_i\| + \|\mathbf{T}\mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w})\| \leq \sqrt{mdN}\sigma_r + a_2 \leq (\sqrt{mdN} + 1)\sigma_r \leq L_1$, where we used $a_2 = \max_{\mathbf{u} \in \mathcal{R}_q^n} \|\mathbf{T}\mathbf{G}^{-1}(\mathbf{u})\|$ and $\sigma_r \geq a_2$. $\square$

4.3 Adaptive Security

We show that Oriole achieves adaptive strong unforgeability (adp-TS-sUF-4) under the MSIS and MLWE assumptions in the random oracle model. We refer to Section 2 for a proof overview. The algorithm SVer is defined in Figure 5, where a signature $(\mathbf{z}, \text{salt})$ is associated with a query $(\mu, \text{SS}, \mathcal{M})$ if and only if $H_2(\mu, \text{SS}, \mathcal{M}) = \text{salt}$. It satisfies the property that it is hard to find two distinct signatures, $(\mathbf{z}, \text{salt})$ and $(\mathbf{z}', \text{salt}')$, such that they are valid and associated with the same query $(\mu, \text{SS}, \mathcal{M})$. Intuitively, if the adversary can forge two such signatures, it holds that $\text{salt} = H_2(\mu, \text{SS}, \mathcal{M}) = \text{salt}'$ and $\mathbf{A}\mathbf{z} = H_1(\mu, \text{salt}) = H_1(\mu, \text{salt}') = \mathbf{A}\mathbf{z}'$. Since $\mathbf{z} \neq \mathbf{z}'$, we can extract an MSIS solution $\mathbf{z} - \mathbf{z}'$ for the matrix $\mathbf{A}$. This property holds even if the adversary can make signing queries, where the reduction can use the same simulation strategy as in the strong unforgeability proof to answer the signing queries.

Proof structure. The proof follows a two-step approach, similar to the prior works [ZT25, JTWZ26]. We consider an intermediate security game, referred to as the adp-IUF game (defined in Fig. 6), where-in for the second-round message corresponding to oSIGN_2 , the adversary only sees the partial sum of partial signatures, i.e., $\hat{\mathbf{z}}$ in Equation (2), instead of the individual partial signatures. Note that the oracle oSIGN_2 now takes as input $(\mu, \text{SS}, \mathcal{M})$ instead of $(i, \mu, \text{SS}, \mathcal{M})$. Also, each signer no longer stores a secret key in its local state. The rest of the game is the same as the adp-TS-sUF-4 game.

- In the first step (Theorem 2), we reduce the adp-TS-sUF-4 game of the threshold scheme to the adp-IUF game. This step is similar to the previous proofs in [ZT25, JTWZ26] (which in turn build on [EKT24]), and uses masking in an essential way. The proof is deferred to Section 4.6.
- In the second step (Theorem 3), we establish the hardness of the adp-IUF game based on the HNF-MSIS and MLWE assumptions. This step is where the main novelty lies.

Game $\text{adp-IUF}^A(\kappa)$: $(\mathbf{A}, \mathbf{T}) \leftarrow \text{KeyGen}(1^\kappa)$ $\text{pk} = \mathbf{A} ; \text{sk} = \mathbf{T}$ $\text{TrivQ} = \emptyset ; \text{CS} = \emptyset$ For $i \in [N]$: $\quad \text{st}_i.\text{mapPP} = ()$ $\quad \text{st}_i.\text{validPP} = ()$ $\quad (\mu^*, \text{sig}^*) \leftarrow \mathcal{A}^{\text{OSIGN}_1, \text{OSIGN}_2, \text{RO}, \text{COR}}(\text{pk})$ $\quad (\mathbf{z}^*, \text{salt}^*) = \text{sig}^*$ If $\exists (\mu, \text{SS}, \mathcal{M}) \in \text{TrivQ}$: $\quad \mu^* = \mu \wedge \text{salt}^* = H_2(\mu, \text{SS}, \mathcal{M})$ then $\quad \quad \text{Return } 0$ Return $\text{Ver}(\text{pk}, \mu, \text{sig}) = 1$ Oracle $\text{COR}(i)$: Require: $i \in [n] \setminus \text{CS}$ and $ \text{CS} < T - 1$ $\text{CS} = \text{CS} \cup \{i\}$ Return st_i Oracle $\text{RO}(i, x)$: Return $H_i(x)$	Oracle $\text{OSIGN}_1(i)$: Require: $i \in [N] \setminus \text{CS}$ $\mathbf{r}_i \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_r}, \mathbf{w}_i = \mathbf{A}\mathbf{r}_i$ $\text{st}_i.\text{mapPP}(\mathbf{w}_i) = \mathbf{r}_i$ $\text{st}_i.\text{validPP}(\mathbf{w}_i) = 1$ Return $\mathbf{w}_i$ Oracle $\text{OSIGN}_2(\mu, \text{SS}, \mathcal{M})$: Require: $\text{SS} \subseteq [N]$ and $ \text{SS} \geq T$ $\{(i, \mathbf{w}_i)\}_{i \in \text{SS}} = \mathcal{M}$ If $\exists i \in \text{SS} \setminus \text{CS}$: $\quad \text{st}_i.\text{validPP}(\mathbf{w}_i) \neq 1$ then $\quad \quad \text{Return } \perp$ $\text{TrivQ} = \text{TrivQ} \cup \{(\mu, \text{SS}, \mathcal{M})\}$ $\mathbf{w} = \sum_{i \in \text{SS}} \mathbf{w}_i$ $\text{salt} = H_2(\mu, \text{SS}, \mathcal{M})$ For $i \in \text{SS} \setminus \text{CS}$: $\quad \mathbf{r}_i = \text{st}_i.\text{mapPP}(\mathbf{w}_i)$ $\quad \text{st}_i.\text{validPP}(\mathbf{w}_i) = 0$ $\mathbf{z} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \mathbf{T}\mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w})$ Return $\mathbf{z}$
---	--

Fig. 6. The adaptive-intermediate game adp-IUF . The algorithm KeyGen is defined in Section 4.4 and $(\mathbf{A}, \mathbf{T})$ satisfies $\mathbf{AT} = \mathbf{G}$, where $\mathbf{A} \in \mathcal{R}_q^{n \times m}$, $\mathbf{T} \in \mathcal{R}_q^{m \times \ell}$, $\mathbf{G} \in \mathcal{R}_q^{n \times \ell}$.

Theorem 2 (From adp-TS-sUF-4 to adp-IUF security). For any integer $q = q(\kappa)$, $n = n(\kappa)$, $m = m(\kappa)$, $\ell = \ell(\kappa)$ and any adp-TS-sUF-4 adversary $\mathcal{A}$ making at most $Q_s = Q_s(\kappa)$ queries to the OSIGN_1 oracle and $Q_h = Q_h(\kappa)$ random oracle queries, there exist an adp-IUF adversary $\mathcal{B}$ making at most Q_s queries to the OSIGN_1 oracle and $Q_h + Q_s$ random oracle queries such that

$$\text{Adv}_{\text{Oriole}}^{\text{adp-ts-suf-4}}(\mathcal{A}, \kappa) \leq \text{Adv}^{\text{adp-IUF}}(\mathcal{B}, \kappa) + Q_h(Q_h + N^2) \cdot 2^{-2\kappa+1},$$

where N denotes the number of signers.

Theorem 3 (From adp-IUF to $\text{HNF-MSIS}'$). For any integers $q = q(\kappa)$, $n = n(\kappa)$, $m = m(\kappa)$, $\ell = \ell(\kappa)$ and any adversary $\mathcal{A}$ making at most Q_s queries to OSIGN_1 oracle and Q_h queries to random oracle, there exists an $\text{HNF-MSIS}'$ adversary $\mathcal{B}$ running in time roughly that of $\mathcal{A}$ such that

$$\begin{aligned} \text{Adv}^{\text{adp-IUF}}(\mathcal{A}, \kappa) \leq & Q_h \cdot (e^\pi \cdot \delta \cdot \sqrt{2\text{Adv}_{q,d,n,k,\ell,\beta_{\text{sis}},\beta_{\text{hwe}}}^{\text{hnf-msis}'}(\mathcal{B}, \kappa)} + Q_h \cdot 2^{-md+2} \\ & Q_s \cdot 2^{-md+1} + 2Q_h \cdot \varepsilon + 2Q_h^2 \cdot 2^{-\lambda}). \end{aligned}$$

where we used $\sigma_1 \geq 2\eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))$, $\sigma_0 \geq \eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))$, $\sigma_r \geq \max\{\sqrt{Q} \cdot (a_1 + a_2), 2\eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))\}$, $\varepsilon = 2^{-2\kappa}$, $Q = 2Q_s + 2$, $a_2 = \max_{\mathbf{u} \in \mathcal{R}_q^n} \|\mathbf{T}\mathbf{G}^{-1}(\mathbf{u})\|$, $a_1 = \sigma_0 \cdot \sqrt{md}$, and $\delta = \sqrt{(1 + 16N \cdot Q \cdot 2^{-2\kappa})(1 + 4(2Q + 1) \cdot 2^{-2\kappa})}$.

Combined with Lemma 7, this establishes security under the MSIS and MLWE assumptions.

Proof (of Theorem 3). Let $\mathcal{B}$ be an adversary against $\text{HNF-MSIS}'$ problem, and $\mathcal{A}$ be an adversary against adp-IUF game. Given an $\text{HNF-MSIS}'$ instance $\mathbf{A} \in \mathcal{R}_q^{n \times m}$, $\mathcal{B}^{\mathcal{A}}(\mathbf{A})$ works as in Figure 7.

W.l.o.g., we make the following simplifying assumptions about $\mathcal{A}$:

- Whenever $\mathcal{A}$ makes a query to OSIGN_2 , we assume that it has already queried H_1 and H_2 on the corresponding inputs used by that algorithm. If not, we simply make the missing queries on its behalf.

<u>Algorithm $\mathcal{B}^{\mathcal{A}}(\mathbf{A})$:</u> $\text{pk} = \mathbf{A}$; $\text{setSalt} = \emptyset$; $\text{CS} = \emptyset$; $\text{cnt} = 0$ $\text{TrivQ} = \emptyset$ $j_1 \leftarrow [Q_h]$ Sample $\mathbf{z}_0 \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_1}$ For $i \in [N]$: $\text{st}_i.\text{mapPP} = ()$; $\text{st}_i.\text{validPP} = ()$ $(\mu^*, \text{sig}^*) \leftarrow \mathcal{A}^{\text{osIGN}_1, \text{osIGN}_2, H_1, H_2, \text{COR}}(\text{pk})$ $(\mathbf{z}^*, \text{salt}^*) = \text{sig}^*$ If $\text{iforgery} \neq (\mu^*, \text{salt}^*)$, return $\perp$ If $\exists (\mu, \text{SS}, \mathcal{M}) \in \text{TrivQ} : \mu^* = \mu$ $\wedge \text{salt}^* = H_2(\mu, \text{SS}, \mathcal{M})$ then Return $\perp$ Return $\mathbf{z}_0 - \mathbf{z}^*$ <u>Oracle $\text{COR}(i)$:</u> Require: $i \in [n] \setminus \text{CS}$ and $\text{CS} < T - 1$ $\text{CS} = \text{CS} \cup \{i\}$ Return st_i <u>Oracle $H_2(x)$:</u> If $\text{mapH}_2(x) \neq \perp$, return $\text{mapH}_2(x)$ Sample $\text{salt} \leftarrow \{0, 1\}^\lambda \setminus \text{setSalt}$ $\text{mapH}_2(x) = \text{salt}$ $\text{setSalt} = \text{setSalt} \cup \{\text{salt}\}$ Parse $x = \{\mu, \text{SS}, \mathcal{M}\}$, $\{(i, \mathbf{w}_i)\}_{i \in \text{SS}} = \mathcal{M}$ Sample $\tilde{\mathbf{y}} \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_0}$ $\text{mapH}_1(\mu, \text{salt}) = \mathbf{A}\tilde{\mathbf{y}} + \sum_{i \in \text{SS}} \mathbf{w}_i$ $\text{auxH}_1(\mu, \text{salt}) = \tilde{\mathbf{y}}$ Return $\text{mapH}_2(x)$	<u>Oracle $H_1(x)$:</u> If $\text{cnt} = j_1$ then $\text{mapH}_1(x) = \mathbf{A}\mathbf{z}_0$ $\text{iforgery} = x$ $\text{cnt} = \text{cnt} + 1$ If $\text{mapH}_1(x) = \perp$ then Sample $\mathbf{u} \leftarrow \mathcal{R}_q^n$ $\text{mapH}_1(x) = \mathbf{u}$ $\text{cnt} = \text{cnt} + 1$ Parse $x = \{\mu, \text{salt}\}$ $\text{setSalt} = \text{setSalt} \cup \{\text{salt}\}$ Return $\text{mapH}_1(x)$ <u>Oracle $\text{osIGN}_1(i)$:</u> Require: $i \in [N] \setminus \text{CS}$ $\mathbf{r}_i \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_r}$, $\mathbf{w}_i = \mathbf{A}\mathbf{r}_i$ $\text{st}_i.\text{mapPP}(\mathbf{w}_i) = \mathbf{r}_i$ $\text{st}_i.\text{validPP}(\mathbf{w}_i) = 1$ Return $\mathbf{w}_i$ <u>Oracle $\text{osIGN}_2(\mu, \text{SS}, \mathcal{M})$:</u> Require: $\text{SS} \geq T$ $\{(i, \mathbf{w}_i)\}_{i \in \text{SS}} = \mathcal{M}$ If $\exists i \in \text{SS} \setminus \text{CS}$: $\text{st}_i.\text{validPP}(\mathbf{w}_i) \neq 1$ then Return $\perp$ $\text{TrivQ} = \text{TrivQ} \cup \{(\mu, \text{SS}, \mathcal{M})\}$ For $i \in \text{SS} \setminus \text{CS}$: $\mathbf{r}_i = \text{st}_i.\text{mapPP}(\mathbf{w}_i)$ $\text{st}_i.\text{validPP}(\mathbf{w}_i) = 0$ $\text{salt} = H_2(\mu, \text{SS}, \mathcal{M})$ $\tilde{\mathbf{y}} = \text{auxH}_1(\mu, \text{salt})$ $\hat{\mathbf{z}} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \tilde{\mathbf{y}}$ Return $\hat{\mathbf{z}}$
--	---

Fig. 7. The adversary $\mathcal{B}$ against HNF-MSIS' that runs $\mathcal{A}$ by simulating IUF game.

- $\mathcal{A}$ never repeats a query to H_1 or to H_2 ;
- $\mathcal{A}$ always queries H_1 on (μ^*, salt^*) corresponding to the output signature $(\mu^*, \text{salt}^*, \mathbf{z}^*)$ (if it doesn't, just make an extra query).

We proceed via a sequence of games, and write Adv_x to denote the advantage in G_x .

- G_0 : Same as the original adp-IUF game defined in Fig. 6, let $(\mu^*, \text{sig}^* = (\text{salt}^*, \mathbf{z}^*))$ denote the message-signature pair output by the adversary $\mathcal{A}$.
- G_1 : Same as G_0 , except we sample $j_1 \leftarrow [Q_h]$ at the beginning of the game and abort at the end if the j_1 'th query to H_1 is not (μ^*, salt^*) . We have

$$\text{Adv}_1 = \frac{1}{Q_h} \cdot \text{Adv}_0.$$

The differences are shown in Figure 8.

- G_2 : Same as G_1 , except that we change the way in which the outputs of H_2 are generated.
- G_2 maintains a set setSalt that records all salt values that have appeared so far, including all outputs previously returned by H_2 and all salt values occurring in H_1 queries up to the current point in the game. It also maintains

a table mapH_2 , where the key of each entry is an oracle query x to H_2 and the entry stores the corresponding response value $H_2(x)$.

Concretely, whenever H_2 returns a new salt, or the adversary queries H_1 on an input of the form (μ, salt) , we add salt to setSalt .

Upon receiving a fresh query $(\mu, \text{SS}, \mathcal{M})$ to H_2 , we sample a new salt salt' uniformly at random from $\{0, 1\}^\lambda \setminus \text{setSalt}$, return salt' , and add it to setSalt . Let Collision-salt denote the event that H_2 in G_1 outputs some $\text{salt}' \in \text{setSalt}$. Since the output space of H_2 is $\{0, 1\}^\lambda$, we have $\Pr[\text{Collision-salt}] \leq Q_h^2 \cdot 2^{-\lambda}$.

Therefore,

$$|\text{Adv}_1 - \text{Adv}_2| \leq Q_h^2 \cdot 2^{-\lambda}.$$

The detailed differences are shown in Figure 8.

G₃: Same as G_2 , except that we modify the way in which the outputs of H_1 are generated. In particular, during the evaluation of H_2 we program certain values of H_1 and compute auxiliary information that will later be used to simulate $\text{oSIGN}_2(\mu, \text{SS}, \mathcal{M})$.

Compared to G_2 , G_3 maintains two additional tables, mapH_1 and auxH_1 . For mapH_1 , the key of each entry is an oracle query x to H_1 , and the entry stores the corresponding response value $H_1(x)$. For auxH_1 , the key of each entry is a pair (μ, salt) . Such an entry is created whenever a query $(\mu, \text{SS}, \mathcal{M})$ is made to H_2 (with output salt), and it stores the auxiliary value $\tilde{\mathbf{y}}$, where $\tilde{\mathbf{y}} \in \mathcal{R}_q^m$.

Concretely, for every fresh query of the form $(\mu, \text{SS}, \mathcal{M})$ to H_2 , where $\mathcal{M} = \{(i, \mathbf{w}_i)\}_{i \in \text{SS}}$, let salt denotes the output. We sample $\tilde{\mathbf{y}} \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_0}$ and define

$$\text{mapH}_1(\mu, \text{salt}) = \mathbf{A}\tilde{\mathbf{y}} + \sum_{i \in \text{SS}} \mathbf{w}_i,$$

which serves as a candidate value for $H_1(\mu, \text{salt})$ (except for the j_1 -th query to H_1). In addition, we set

$$\text{auxH}_1(\mu, \text{salt}) = \tilde{\mathbf{y}},$$

which will be used in $\text{oSIGN}_2(\mu, \text{SS}, \mathcal{M})$ in a later game.

For each fresh query to H_1 , if it is the j_1 -th query (μ^*, salt^*) , we sample $\mathbf{z}_0 \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_1}$, program $H_1(\mu^*, \text{salt}^*) = \mathbf{A}\mathbf{z}_0$, and return this value. For any other input x , if $\text{mapH}_1(x) \neq \perp$, we return $\text{mapH}_1(x)$. Otherwise, if $\text{mapH}_1(x) = \perp$, we sample a uniform $\mathbf{u} \leftarrow \mathcal{R}_q^n$, program $H_1(x) = \text{mapH}_1(x) = \mathbf{u}$, and return it.

By Lemma 4, if $\sigma_0, \sigma_1 \geq \eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))$, then both $\mathbf{A}\tilde{\mathbf{y}}$ and $\mathbf{A}\mathbf{z}_0$ are statistically close to the uniform distribution over $\mathcal{R}_q^n$. Therefore, we have

$$|\text{Adv}_2 - \text{Adv}_3| \leq 2Q_h \cdot \varepsilon.$$

The detailed differences between the games are summarized in Figure 9.

G₄: Same as G_3 , except that we modify the way in which the outputs of oSIGN_2 are generated.

Let $(\mu, \text{SS}, \mathcal{M})$ be a query to oSIGN_2 , where $\mathcal{M} = \{(i, \mathbf{w}_i)\}_{i \in \text{SS}}$. If for all $i \in \text{SS} \setminus \text{CS}$ we have $\text{st}_i.\text{validPP}(\mathbf{w}_i) = 1$, then instead of returning $\mathbf{z} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \mathbf{T}\mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w})$ as the output of $\text{oSIGN}_2(\mu, \text{SS}, \mathcal{M})$, we return

$$\hat{\mathbf{z}} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \tilde{\mathbf{y}},$$

where $\mathbf{r}_i = \text{st}_i.\text{mapPP}(\mathbf{w}_i)$, $\tilde{\mathbf{y}} = \text{auxH}_1(\mu, \text{salt})$, and $\text{salt} = H_2(\mu, \text{SS}, \mathcal{M})$. We relate Adv_3 and Adv_4 in Lemma 11, and the detailed changes between the games are summarized in Figure 9.

Analysis of $\mathcal{B}$ We now construct an adversary $\mathcal{B}$ playing the HNF-MSIS' game in Figure 7 that runs $\mathcal{A}$ by simulating the G_4 game.

It's clear that $\mathcal{B}$ simulates the oracles perfectly. If the adversary $\mathcal{A}$ in G_4 outputs a valid forged signature $(\mu^*, \text{salt}^*, \mathbf{z}^*)$ such that $\mathbf{A}\mathbf{z}^* = H_1(\mu^*, \text{salt}^*)$ and $\|\mathbf{z}^*\| \leq L_1$, then we also have $\mathbf{A}\mathbf{z}_0 = H_1(\mu^*, \text{salt}^*)$, where $\mathbf{z}_0 \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_1}$. By Lemma 2, if $\sigma_1 \geq 2 \cdot \eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))$, we obtain

$$\Pr[\mathbf{z}_0 \neq \mathbf{z}^*] \geq 1 - 2^{-(m-1)d} \geq \frac{1}{2}.$$

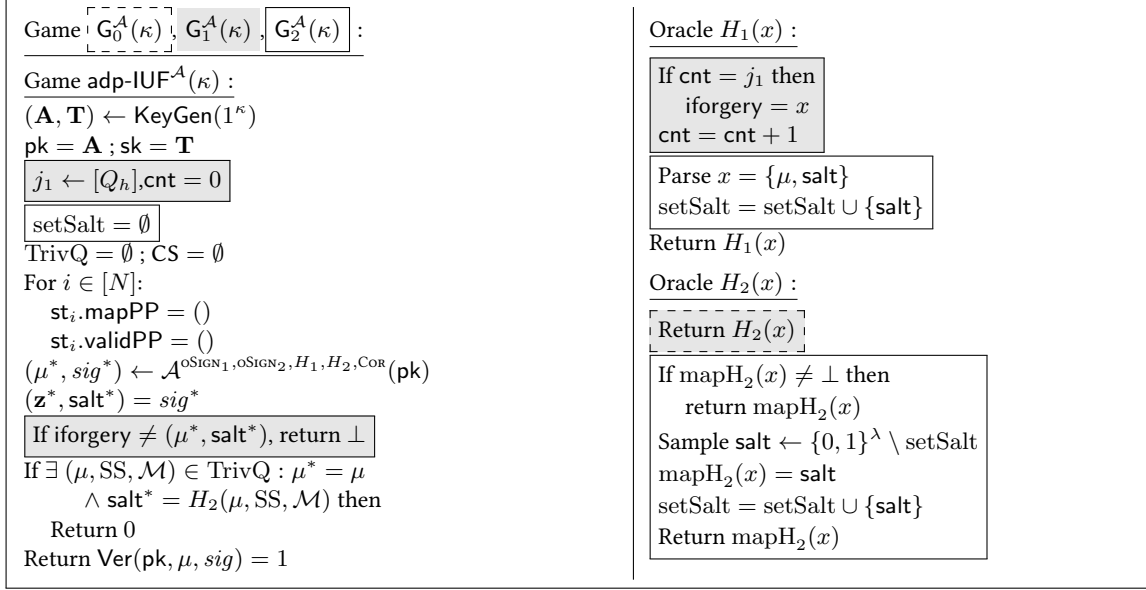


Fig. 8. Initialization and the oracles H_1, H_2 of the games G_0, G_1 , and G_2 , where G_0 only contains the dashed box, G_1 only contains the highlighted boxes, and G_2 only contains the solid boxes. G_1 and G_2 maintain a counter cnt that tracks the number of queries made to H_1 , and they record the input of the j_1 -th query to H_1 as iforgery . In addition, G_2 maintains a set setSalt that stores all salt values that have appeared so far, including all outputs previously returned by H_2 and all salt values occurring in H_1 queries up to the current point in the game. The rest of each game is the same as Fig. 6

Moreover, the conditions under which $\mathcal{B}$ wins are that $\mathbf{A}(\mathbf{z}_0 - \mathbf{z}^*) = \mathbf{0}$ and

$$0 < \|\mathbf{z}_0 - \mathbf{z}^*\| \leq \sigma_1 \sqrt{md} + L_1 \leq 2L_1 = \beta_{\text{sis}}.$$

Therefore, by Lemma 1, the winning probability of $\mathcal{B}$ is at least $\frac{1}{2} \text{Adv}_4(\mathcal{A}, \kappa) - \frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-md}$. Thus, we conclude that

$$\text{Adv}_{q,d,n,m,\beta_{\text{sis}}}^{\text{hnf-msis}'}(\mathcal{B}, \kappa) \geq \frac{1}{2} \text{Adv}_4(\mathcal{A}, \kappa) - 2^{-md+1}.$$

□

The following lemma establishes the relationship between Adv_3 and Adv_4 . It relies on the conditional distribution described in Lemma 6 to characterize the information revealed when the adversary corrupts a signer or issues a query to OSIGN_2 . By combining the noise-flooding technique of Lemma 10 with the weak triangle inequality from Lemma 8, We show that the Rényi distance between the transcript distributions generated in the two games is small.

Lemma 11 (Bounding Adv_3). *Given $\sigma_r \geq \max\{\sqrt{Q} \cdot (a_1 + a_2), 2\eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))\}$, where $a_1 = \sigma_0 \cdot \sqrt{md}$ and $a_2 = \max_{\mathbf{u} \in \mathcal{R}_q^n} \|\mathbf{T}\mathbf{G}^{-1}(\mathbf{u})\|$,*

$$\text{Adv}_3(\mathcal{A}, \kappa) \leq e^\pi \cdot \delta \cdot \text{Adv}_4^{\frac{1}{2}}(\mathcal{A}, \kappa) + Q_s \cdot \frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-md}.$$

where $\delta = \sqrt{(1 + 16N \cdot Q \cdot 2^{-2\kappa})(1 + 4(2Q + 1) \cdot 2^{-2\kappa})}$ and $Q = 2Q_s + 2$.

Proof. To maintain consistency, we require that for every query $(\mu, \text{SS}, \{(i, \mathbf{w}_i)\}_{i \in \text{SS}})$ to OSIGN_2 , if for all $i \in \text{SS} \setminus \text{CS}$ it holds that $\text{st}_i.\text{mapPP}(\mathbf{w}_i) = 1$ and $\text{auxH}_1(\mu, \text{salt}) = \tilde{\mathbf{y}}$, then the value $\hat{\mathbf{z}} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \tilde{\mathbf{y}}$ returned by the OSIGN_2 satisfies the expected linear relation. Concretely, recalling that $H_1(\mu, \text{salt}) - \sum_{i \in \text{SS}} \mathbf{w}_i = \mathbf{A}\tilde{\mathbf{y}}$, we obtain

$$\mathbf{A}\hat{\mathbf{z}} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{w}_i + \mathbf{A}\tilde{\mathbf{y}} = H_1(\mu, \text{salt}) - \sum_{i \in \text{CS}} \mathbf{w}_i = \mathbf{A}\mathbf{z},$$

Game $G_2^A(\kappa)$, $G_3^A(\kappa)$, $G_4^A(\kappa)$: Game $\text{adp-IUF}^A(\kappa)$: $(\mathbf{A}, \mathbf{T}) \leftarrow \text{KeyGen}(1^\kappa)$ $\text{pk} = \mathbf{A}; \text{sk} = \mathbf{T}$ $(\mathbf{A}, \mathbf{T}) \leftarrow \text{KeyGen}(1^\kappa)$ $\text{pk} = \mathbf{A}; \text{sk} = \perp$ Sample $\mathbf{z}_0 \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_1}$ $\text{cnt} = 0; \text{setSalt} = \emptyset$ $j_1 \leftarrow [Q_h]$ $\text{TrivQ} = \emptyset; \text{CS} = \emptyset$ For $i \in [N]$: $\text{st}_i.\text{mapPP} = ()$ $\text{st}_i.\text{validPP} = ()$ $(\mu^*, \text{sig}^*) \leftarrow \mathcal{A}^{\text{oSIGN}_2, H_1, H_2, \text{COR}}(\text{pk})$ $(\mathbf{z}^*, \text{salt}^*) = \text{sig}^*$ If $\text{iforgery} \neq (\mu^*, \text{salt}^*)$, return $\perp$ If $\exists (\mu, \text{SS}, \mathcal{M}) \in \text{TrivQ} : \mu^* = \mu$ $\wedge \text{salt}^* = H_2(\mu, \text{SS}, \mathcal{M})$ then Return 0 Return $\text{Ver}(\text{pk}, \mu, \text{sig}) = 1$ Oracle $H_2(x)$: If $\text{mapH}_2(x) \neq \perp$ then return $\text{mapH}_2(x)$ Sample $\text{salt} \leftarrow \{0, 1\}^\lambda \setminus \text{setSalt}$ $\text{mapH}_2(x) = \text{salt}$ $\text{setSalt} = \text{setSalt} \cup \{\text{salt}\}$ Parse $x = \{\mu, \text{SS}, \mathcal{M}\}$ and $\{(i, \mathbf{w}_i)\}_{i \in \text{SS}} = \mathcal{M}$ Sample $\tilde{\mathbf{y}} \leftarrow \mathcal{D}_{\mathcal{R}^m, \sigma_0}$ $\text{mapH}_1(\mu, \text{salt}) = \mathbf{A}\tilde{\mathbf{y}} + \sum_{i \in \text{SS}} \mathbf{w}_i$ $\text{auxH}_1(\mu, \text{salt}) = \tilde{\mathbf{y}}$ Return $\text{mapH}_2(x)$	Oracle $\text{COR}(i)$: Require: $i \in [n] \setminus \text{CS}$ and $\text{CS} < T - 1$ $\text{CS} = \text{CS} \cup \{i\}$ Return st_i Oracle $H_1(x)$: If $\text{cnt} = j_1$ then $\text{mapH}_1(x) = \mathbf{A}\mathbf{z}_0$ $\text{iforgery} = x$ $\text{cnt} = \text{cnt} + 1$ If $\text{mapH}_1(x) = \perp$ then Sample $\mathbf{u} \leftarrow \mathcal{R}_q^n$ $\text{mapH}_1(x) = \mathbf{u}$ $\text{cnt} = \text{cnt} + 1$ Parse $x = \{\mu, \text{salt}\}$ $\text{setSalt} = \text{setSalt} \cup \{\text{salt}\}$ Return $H_1(x)$ Return $\text{mapH}_1(x)$ Oracle $\text{oSIGN}_2(\mu, \text{SS}, \mathcal{M})$: Require: $\text{SS} \subseteq [N]$ and $\text{SS} \geq T$ $\{(i, \mathbf{w}_i)\}_{i \in \text{SS}} = \mathcal{M}$ If $\exists i \in \text{SS} \setminus \text{CS}$: $\text{st}_i.\text{validPP}(\mathbf{w}_i) \neq 1$ then Return $\perp$ $\text{TrivQ} = \text{TrivQ} \cup \{(\mu, \text{SS}, \mathcal{M})\}$ For $i \in \text{SS} \setminus \text{CS}$: $\mathbf{r}_i = \text{st}_i.\text{mapPP}(\mathbf{w}_i)$ $\text{st}_i.\text{validPP}(\mathbf{w}_i) = 0$ $\text{salt} = H_2(\mu, \text{SS}, \mathcal{M})$ $\mathbf{w} = \sum_{i \in \text{SS}} \mathbf{w}_i$ $\mathbf{z} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i +$ $\mathbf{TG}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w})$ Return $\mathbf{z}$ $\tilde{\mathbf{y}} = \text{auxH}_1(\mu, \text{salt})$ $\hat{\mathbf{z}} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \tilde{\mathbf{y}}$ Return $\hat{\mathbf{z}}$
--	---

Fig. 9. Initialization, the oracles H_1 , H_2 and oSIGN_2 of the games, G_2 , G_3 , G_4 , where G_2 only contains the dashed box, G_3 only contains the highlighted boxes, and G_4 only contains the solid boxes. The rest of each game is the same as G_2 .

where $\mathbf{z} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \mathbf{TG}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w})$.

To bound Adv_3 , by the data-processing inequality and probability preservation in Lemma 8, it suffices to bound

$$R_\alpha(P_1^{(0)}, \dots, P_Q^{(0)} \| P_1^{(1)}, \dots, P_Q^{(1)}),$$

where $Q = 2Q_s + 2$ is an upper bound on the total number of queries that the adversary can make to the oSIGN_2 and COR oracles in the adp-IUF game of Fig. 6. Here, $P_i^{(0)}$ (resp. $P_i^{(1)}$) denotes the transcript received by the adversary in G_3 (resp. G_4). Each $P_i^{(0)}$ (resp. $P_i^{(1)}$) is either a value of the form $\mathbf{z} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \mathbf{TG}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w})$ (resp.

$\hat{\mathbf{z}} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \tilde{\mathbf{y}}$ returned by $\text{oSIGN}_2(\mu, \text{SS}, \{(i, \mathbf{w}_i)\}_{i \in \text{SS}})$ with $\text{salt} = H_2(\mu, \text{SS}, \{(i, \mathbf{w}_i)\}_{i \in \text{SS}})$, or a value $\mathbf{r}_i$ returned by $\text{COR}(i)$.

If the adversary only issues queries to oSIGN_2 , the argument follows directly from the noise-flooding technique of Lemma 10. The main technical challenge arises from the fact that the adversary may additionally obtain some $\mathbf{r}_i$ via queries to $\text{COR}(i)$, and that the indices i can be chosen adaptively based on the previously observed values $\mathbf{z}(\text{resp. } \hat{\mathbf{z}})$ and $\mathbf{r}_i$.

By Lemma 1, except with probability at most $Q_s \cdot \frac{1+\varepsilon}{1-\varepsilon} \cdot 2^{-md}$, we have $\|\tilde{\mathbf{y}}\| \leq \sigma_0 \cdot \sqrt{md} = a_1$ and $\|\mathbf{TG}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w})\| \leq a_2 := \max_{\mathbf{u} \in \mathcal{R}_q^n} \|\mathbf{TG}^{-1}(\mathbf{u})\|$. Moreover, by Lemma 9, it holds that

$$R_\alpha(P_1^{(0)}, \dots, P_Q^{(0)} \| P_1^{(1)}, \dots, P_Q^{(1)}) \leq \prod_{i \in [Q]} \max_{x_{[i-1]}} R_\alpha(P_{i|x_{[i-1]}}^{(0)} \| P_{i|x_{[i-1]}}^{(1)}).$$

If $P_i^{(0)}$ (resp. $P_i^{(1)}$) is $\mathbf{z} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \mathbf{TG}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w})$ (resp. $\hat{\mathbf{z}} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \tilde{\mathbf{y}}$), let $t' = |\text{SS} \setminus \text{CS}| \leq N$, then by Lemma 6 we have $P_{i|x_{[i-1]}}^{(0)} \stackrel{\varepsilon'}{\approx} \mathcal{D}_{\Lambda_q^\perp(\mathbf{A}) + \mathbf{a}, \sqrt{t'} \cdot \sigma_r, \mathbf{c}_0} =: \tilde{P}_{i|x_{[i-1]}}^{(0)}$ and $P_{i|x_{[i-1]}}^{(1)} \stackrel{\varepsilon'}{\approx} \mathcal{D}_{\Lambda_q^\perp(\mathbf{A}) + \mathbf{a}, \sqrt{t'} \cdot \sigma_r, \mathbf{c}_1} =: \tilde{P}_{i|x_{[i-1]}}^{(1)}$, where $\mathbf{c}_0 = \mathbf{TG}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w})$, $\mathbf{c}_1 = \tilde{\mathbf{y}}$, $\varepsilon' = \frac{2\varepsilon}{1-\varepsilon}$ and the $\mathbf{a}$ is a vector satisfies $\mathbf{A}\mathbf{a} = H_1(\mu, \text{salt})$. Thus by Lemma 10, we have

$$\begin{aligned} R_\alpha(\tilde{P}_{i|x_{[i-1]}}^{(0)} \| \tilde{P}_{i|x_{[i-1]}}^{(1)}) &\leq \left(\frac{1+\varepsilon}{1-\varepsilon}\right)^{\frac{\alpha}{\alpha-1}} \exp(\alpha\pi \frac{\|\mathbf{c}_0 - \mathbf{c}_1\|^2}{\sqrt{t'} \sigma_r^2}) \\ &\leq \left(\frac{1+\varepsilon}{1-\varepsilon}\right)^{\frac{\alpha}{\alpha-1}} \cdot \exp(\alpha\pi \cdot \frac{(a_1 + a_2)^2}{\sigma_r^2}). \end{aligned}$$

Here we need $\sigma_r \geq 2\eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))$. And by the weak triangle inequalities from Lemma 8, we have

$$\begin{aligned} R_\alpha(P_{i|x_{[i-1]}}^{(0)} \| P_{i|x_{[i-1]}}^{(1)}) &\leq R_\infty(P_{i|x_{[i-1]}}^{(0)} \| \tilde{P}_{i|x_{[i-1]}}^{(0)})^{\frac{\alpha}{\alpha-1}} \cdot R_\alpha(\tilde{P}_{i|x_{[i-1]}}^{(0)} \| \tilde{P}_{i|x_{[i-1]}}^{(1)}) \\ &\quad \cdot R_\infty(\tilde{P}_{i|x_{[i-1]}}^{(1)} \| P_{i|x_{[i-1]}}^{(1)}) \\ &\leq (1 + \varepsilon')^{\frac{2\alpha-1}{\alpha-1}} \cdot \left(\frac{1+\varepsilon}{1-\varepsilon}\right)^{\frac{\alpha}{\alpha-1}} \cdot \exp(\alpha\pi \cdot \frac{(a_1 + a_2)^2}{\sigma_r^2}). \end{aligned}$$

If $P_i^{(0)}$ (resp. $P_i^{(1)}$) is $\mathbf{r}_i$, and $\mathbf{w}_i = \mathbf{A}\mathbf{r}_i$, where $\mathbf{r}_i$ is in some previous $\mathbf{z} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \mathbf{TG}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w})$ (or in $\hat{\mathbf{z}} = \sum_{i \in \text{SS} \setminus \text{CS}} \mathbf{r}_i + \tilde{\mathbf{y}}$), where $|\text{SS} \setminus \text{CS}| = t'$, and assuming this is the k -th $\mathbf{r}_i$ opened in the $\text{SS} \setminus \text{CS}$, where $k \leq t' - 1$. By Lemma 6, we have $P_{i|x_{[i-1]}}^{(0)} \stackrel{\varepsilon''}{\approx} \mathcal{D}_{\Lambda_q^\perp(\mathbf{A}) + \mathbf{s}_i, V_{11}^{-1}, \mathbf{c}_i^{(0)}}$ and $P_{i|x_{[i-1]}}^{(1)} \stackrel{\varepsilon''}{\approx} \mathcal{D}_{\Lambda_q^\perp(\mathbf{A}) + \mathbf{s}_i, V_{11}^{-1}, \mathbf{c}_i^{(1)}}$, where $V_{11} = \frac{1}{\sigma_r^2} \cdot (1 + \frac{1}{t'-k})$, $\mathbf{c}_i^{(0)} - \mathbf{c}_i^{(1)} = \frac{1}{t'-k+1} \cdot (\mathbf{TG}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) - \tilde{\mathbf{y}})$, and $\varepsilon'' = \frac{2((1+\varepsilon)^{t'-k-1} - 1)}{2 - (1+\varepsilon)^{t'-k-1}}$, the $\mathbf{s}_i$ is a vector satisfies $\mathbf{A}\mathbf{s}_i = \mathbf{w}_i$. Note that we have $\|\mathbf{c}_i^{(0)} - \mathbf{c}_i^{(1)}\| \leq \frac{a_1 + a_2}{2}$, and $V_{11}^{-1} \geq \frac{\sigma_r^2}{2}$.

Similarly, by Lemma 10 and weak triangle Inequality from Lemma 8, we have

$$R_\alpha(P_{i|x_{[i-1]}}^{(0)} \| P_{i|x_{[i-1]}}^{(1)}) \leq (1 + \varepsilon'')^{\frac{2\alpha-1}{\alpha-1}} \cdot \left(\frac{1+\varepsilon}{1-\varepsilon}\right)^{\frac{\alpha}{\alpha-1}} \cdot \exp(\alpha\pi \cdot \frac{(a_1 + a_2)^2}{2\sigma_r^2}).$$

Thus by Lemma 9, we have

$$R_\alpha(P_1^{(0)}, \dots, P_Q^{(0)} \| P_1^{(1)}, \dots, P_Q^{(1)}) \leq (1 + \varepsilon'')^{2Q} \cdot \left(\frac{1+\varepsilon}{1-\varepsilon}\right)^{2Q} \cdot \exp(\alpha\pi Q \cdot \frac{(a_1 + a_2)^2}{\sigma_r^2}).$$

If we set $\alpha = 2$, $\sigma_r \geq \max\{\sqrt{Q} \cdot (a_1 + a_2), 2\eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))\}$, and $\varepsilon = 2^{-2\kappa}$, then $(1 + \varepsilon)^{t'-k-1} \leq 1 + 2(t' - k)\varepsilon$, $\varepsilon'' \leq 8(t' - k) \cdot 2^{-2\kappa} \leq 8N \cdot 2^{-2\kappa}$, so $(1 + \varepsilon'')^{2Q} \leq 1 + 16N \cdot Q \cdot 2^{-2\kappa}$ and $(\frac{1+\varepsilon}{1-\varepsilon})^{2Q} \leq 1 + 4(2Q + 1) \cdot 2^{-2\kappa}$. By Lemma 8, we have

$$\text{Adv}_3(\mathcal{A}, \kappa) \leq e^\pi \cdot \delta \cdot \text{Adv}_{\frac{1}{4}}(\mathcal{A}, \kappa) + Q_s \cdot \frac{1 + \varepsilon}{1 - \varepsilon} \cdot 2^{-md}$$

where $\delta = \sqrt{(1 + 16N \cdot Q \cdot 2^{-2\kappa})(1 + 4(2Q + 1) \cdot 2^{-2\kappa})}$. $\square$

4.4 How to generate $(\mathbf{A}, \mathbf{T})$

In the key generation algorithm, we sample $(\mathbf{A}, \mathbf{T}) \leftarrow \text{KeyGen}(1^\kappa)$ such $\mathbf{A}\mathbf{T} = \mathbf{G}$. Indeed, [MP12] already shows how to do this. Here, we describe a construction that yields better concrete efficiency, following [CGM19, YJW23, EEN⁺24].

- We first sample

$$\mathbf{A}_0 \leftarrow \mathcal{R}_q^{n \times k}, \mathbf{R} \leftarrow \mathcal{B}_{\beta_{\text{lwe}}}^{k \times \ell}, \mathbf{E} \leftarrow \mathcal{B}_{\beta_{\text{lwe}}}^{n \times \ell}$$

and define

$$\mathbf{A} = [\mathbf{I}_n \mid \mathbf{A}_0 \mid \mathbf{A}_0\mathbf{R} + \mathbf{E} + \mathbf{G}_1] \in \mathcal{R}_q^{n \times m},$$

where $\mathbf{G}_1 = \lceil q/p^t \rceil \cdot [\mathbf{I}_n \mid p\mathbf{I}_n \mid \cdots \mid p^{t-1}\mathbf{I}_n] \in \mathcal{R}_q^{n \times tn}$, and $m = k + n + \ell$, $\ell = tn$.

- Next, we define $\mathbf{G} := [\mathbf{I}_n \mid \mathbf{G}_1]$ so that $\mathbf{A} \cdot \overbrace{\begin{bmatrix} \mathbf{I}_n & -\mathbf{E} \\ & -\mathbf{R} \\ & & \mathbf{I}_\ell \end{bmatrix}}^{\mathbf{T}} = \mathbf{G}$.
- Given $\mathbf{u} \in \mathcal{R}_q^n$, we can express $\mathbf{u}$ as $\mathbf{u} = \mathbf{u}_0 + \mathbf{G}_1\mathbf{u}_1$, where $\|\mathbf{u}_0\|_\infty \leq \lceil q/p^t \rceil$, $\|\mathbf{u}_1\|_\infty \leq p$. Furthermore, we have $\|\mathbf{u}_1\| \leq \sqrt{tn}d \cdot p$ and $\|\mathbf{u}_0\| \leq \sqrt{nd} \cdot \lceil q/p^t \rceil$. We define $\mathbf{G}^{-1}(\mathbf{u}) := \begin{bmatrix} \mathbf{u}_0 \\ \mathbf{u}_1 \end{bmatrix}$.

Bounding $\max_{\mathbf{u} \in \mathcal{R}_q^n} \|\mathbf{T}\mathbf{G}^{-1}(\mathbf{u})\|$. We show that $4tnp\beta_{\text{lwe}}\sqrt{kd}^{1.5}$ is an upper bound for $\max_{\mathbf{u} \in \mathcal{R}_q^n} \|\mathbf{T}\mathbf{G}^{-1}(\mathbf{u})\|$. We have

$$\mathbf{A} \cdot \begin{bmatrix} \mathbf{u}_0 - \mathbf{E}\mathbf{u}_1 \\ -\mathbf{R}\mathbf{u}_1 \\ \mathbf{u}_1 \end{bmatrix} = \mathbf{u}, \quad \mathbf{T}\mathbf{G}^{-1}(\mathbf{u}) = \begin{bmatrix} \mathbf{u}_0 - \mathbf{E}\mathbf{u}_1 \\ -\mathbf{R}\mathbf{u}_1 \\ \mathbf{u}_1 \end{bmatrix}.$$

Since $\|\mathbf{R}\|_\infty, \|\mathbf{E}\|_\infty \leq \beta_{\text{lwe}}$, we obtain $\|\mathbf{E}\mathbf{u}_1\|, \|\mathbf{R}\mathbf{u}_1\| \leq tnd \cdot \sqrt{kd} \cdot p \cdot \beta_{\text{lwe}}$. Next, we choose p , t , and q such that:

$$\|\mathbf{u}_0\| \approx \|\mathbf{E}\mathbf{u}_1\| \approx \|\mathbf{R}\mathbf{u}_1\|.$$

To achieve more balanced parameters, we set

$$\sqrt{nd} \cdot \lceil q/p^t \rceil = tnd \cdot \sqrt{kd} \cdot p \cdot \beta_{\text{lwe}}.$$

which implies that

$$p \approx \left(\frac{q}{\beta_{\text{lwe}} t d \sqrt{nk}} \right)^{\frac{1}{t+1}}.$$

Thus, we have

$$\|\mathbf{T}\mathbf{G}^{-1}(\mathbf{u})\| \leq \|\mathbf{u}_0 - \mathbf{E}\mathbf{u}_1\| + \|\mathbf{R}\mathbf{u}_1\| + \|\mathbf{u}_1\| \leq 4 \cdot tnp\beta_{\text{lwe}}\sqrt{kd}^{1.5}.$$

Additional bounds. Finally, we show the upper bound for $\eta_\varepsilon(\Lambda_q^\perp(\mathbf{A}))$ in our setting. By Lemma 3, it suffices to show the upper bound of $\|\tilde{\mathbf{B}}\|$ for some basis $\mathbf{B}$ of lattice $\Lambda_q^\perp(\mathbf{A})$. Let $\mathbf{W} \in \mathcal{R}_q^{(n+\ell) \times k}$ such that $\mathbf{A}_0 + \mathbf{G}\mathbf{W} = \mathbf{0}$, and $\mathbf{S}_\mathbf{G}$ be the standard basis (This is similar to the analysis in Section 4 of [MP12]⁶) of $\Lambda_q^\perp(\mathbf{G})$, and we have $\|\tilde{\mathbf{S}}_\mathbf{G}\| \leq \max\{2p, \lceil q/p^t \rceil + 1\}$, where the second term is dominated. Then we have a basis of $\Lambda_q^\perp(\mathbf{A})$ such that $\mathbf{S}_\mathbf{A} = \begin{bmatrix} \mathbf{I}_n & -\mathbf{E} \\ \mathbf{I}_k & -\mathbf{R} \\ & \mathbf{I}_\ell \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_k \\ \mathbf{W} & \mathbf{S}_\mathbf{G} \end{bmatrix}$, let $\hat{\mathbf{T}} = \begin{bmatrix} \mathbf{I}_n & -\mathbf{E} \\ \mathbf{I}_k & -\mathbf{R} \\ & \mathbf{I}_\ell \end{bmatrix}$, then we have $\|\tilde{\mathbf{S}}_\mathbf{A}\| \leq \sigma_{\max}(\hat{\mathbf{T}}) \cdot \|\tilde{\mathbf{S}}_\mathbf{G}\| \leq \sigma_{\max}(\hat{\mathbf{T}}) \cdot 2t\sqrt{nk}dp\beta_{\text{lwe}}$.

⁶ In [MP12], their $\mathbf{G}$ is of the form $\mathbf{I}_n \otimes (g_1, g_2, \dots, g_\ell)$; in our setting, our $\mathbf{G}$ can be viewed as $(1, \lceil q/p^t \rceil \cdot (1, p, \dots, p^{t-1})) \otimes \mathbf{I}_n$

κ	$\log_2(q)$	n	k	t	p	σ_r	β_{sis}	$ \mathbf{pk} $	$ \text{sig} $	Comm.
128	80	7	7	2	2^{21}	2^{68}	2^{78}	1.00MB	107KB	143KB

Fig. 10. The concrete parameters and estimated efficiency of Oriole for $\kappa = 128$ and $N = 1024$. We set $(d, \beta_{\text{lwe}}) = (512, 8)$. The last column denotes the communication complexity per signer.

For $\sigma_{\max}(\hat{\mathbf{T}})$, the specific parameters are usually determined experimentally, and its order is generally $\beta_{\text{lwe}}(\sqrt{k} + \sqrt{n} + 2\sqrt{\ell})\sqrt{d} \leq 2\beta_{\text{lwe}}\sqrt{md}$ (see the section 5.2 of [MP12]). Substituting this into the upper bound of $\|\tilde{\mathbf{S}}_{\mathbf{A}}\|$, we have:

$$\eta_{\varepsilon}(A_q^{\perp}(\mathbf{A})) \leq \frac{\sqrt{\log(2md(1+1/\varepsilon))}}{\pi} \cdot 4t\sqrt{nmkp}\beta_{\text{lwe}}^2 d^{1.5}. \quad (4)$$

4.5 Parameter selection

Denote $\beta_{\text{lwe}}, \beta_{\text{sis}}$ as the norm of the underlying MLWE and HNF-MSIS assumption, respectively. Given the security parameter κ , the dimension of the ring d , the modulus q , and (n, k, t) defined in Section 4.4, the rest of the parameters are set as follows.

- The number of columns $\mathbf{A}$ is $m = k + n + tn$.
- We set $p = \left\lceil \left(\frac{q}{\beta_{\text{lwe}}td\sqrt{nk}} \right)^{\frac{1}{t+1}} \right\rceil$.
- By Theorem 3, we set $\sigma_r = \max\{\sqrt{2d(Q_s + 1)} \cdot (4tnp\beta_{\text{lwe}}\sqrt{k}d^1 + \sqrt{m} \cdot \eta_{\varepsilon}(A_q^{\perp}(\mathbf{A}))), 2 \cdot \eta_{\varepsilon}(A_q^{\perp}(\mathbf{A}))\}$, where the first term is dominated and $\eta_{\varepsilon}(A_q^{\perp}(\mathbf{A}))$ is estimated according to Equation (4) with $\varepsilon = 2^{-2\kappa}$.
- To guarantee correctness, we set $L_1 = (\sqrt{mdN} + 1)\sigma_r$.

Finally, we optimize over (d, q, n, k, t) subject to the following constraints:

- d is a power of 2 greater than 2κ .
- q is a prime greater than $4\beta_{\text{sis}}$, where $\beta_{\text{sis}} = 2L_1$.
- $\text{Adv}_{q,d,n,m,\beta_{\text{sis}}}^{\text{hnf-msis}}(\mathcal{A}', \kappa) \leq 2^{-\kappa}$, and $\text{Adv}_{q,d,n,n+k,\beta_{\text{lwe}}}^{\text{mlwe}}(C, \kappa) \leq \frac{1}{\ell} \cdot 2^{-\kappa}$.

Optimizations for $\mathbf{z}$. Let $\mathbf{A} = [\mathbf{I}_n \mid \mathbf{A}']$, $\text{sig} = (\text{salt}, \mathbf{z})$ and denote $\mathbf{z} = \begin{pmatrix} \bar{\mathbf{z}} \\ \underline{\mathbf{z}} \end{pmatrix}$, where $\bar{\mathbf{z}} \in \mathcal{R}_q^n$ and $\underline{\mathbf{z}} \in \mathcal{R}_q^{k+\ell}$. Then, the signature only needs to contain $\underline{\mathbf{z}}$ instead of all of $\mathbf{z}$. This is because $\mathbf{A}\mathbf{z} = \bar{\mathbf{z}} + \mathbf{A}'\underline{\mathbf{z}}$. So instead of checking $\mathbf{A}\mathbf{z} = H_1(\mu, \text{salt})$, we simply compute $\bar{\mathbf{z}} := H_1(\mu, \text{salt}) - \mathbf{A}'\underline{\mathbf{z}}$ and check $\sqrt{\|\bar{\mathbf{z}}\|^2 + \|\underline{\mathbf{z}}\|^2} \leq L_1$. Given this optimization, the final signature size is $d(k + tn) \log_2(q)$ bits, and the communication complexity per signer is $d(n + k + tn) \log_2(q)$ bits.

Concrete parameters. We give a candidate concrete parameters in Figure 10. Here we consider $\kappa = 128$, $Q_s = 2^{60}$ and $N = 1024$. The hardness of MLWE and HNF-MSIS is estimated using the state-of-art tool, lattice estimator.⁷

4.6 Proof of Theorem 2

We show the theorem via the following series of games.

G_0 : Same as the $\text{adp-TS-sUF-4}_{\text{Oriole}}$ game defined in Fig 4.

G_1 : Same as G_0 except we change the way how the masks in the oSIGN_2 oracle are computed. We use $\text{mask}_{i,\mu,\text{SS},\mathcal{M}}$ to denote the mask computed by signer i when the adversary makes a second-round signing query $(\mu, \text{SS}, \mathcal{M})$, and we use CompMask to denote the algorithm used to compute the mask. Also, we use a map $\text{curMS}(\mu, \text{SS}, \mathcal{M})$ to record the set of signers that already received the second-round query $(\mu, \text{SS}, \mathcal{M})$.

⁷ <https://github.com/malb/lattice-estimator>

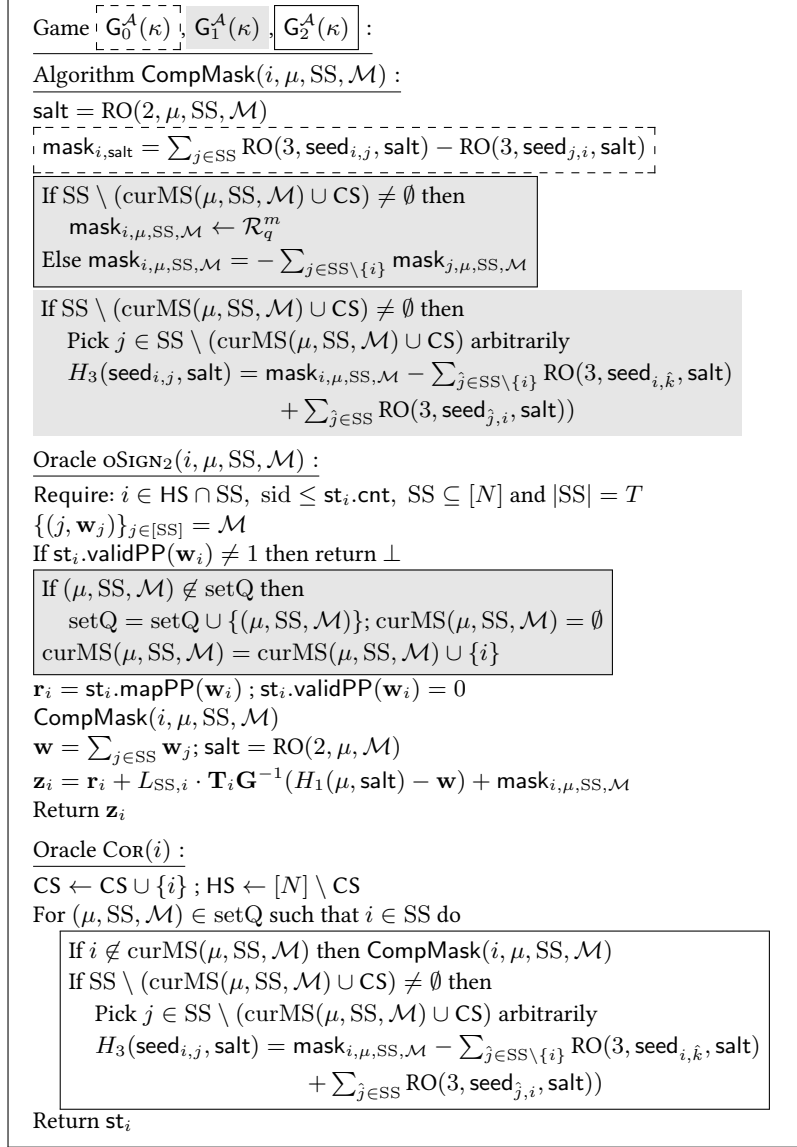


Fig. 11. The oracles oSIGN_2 and COR of the games G_0 , G_1 , and G_2 , where G_0 only contains the dashed box, G_1 only contains the highlighted boxes, and G_2 only contains the solid boxes. The CompMask algorithm takes a second-round query $(\mu, \text{SS}, \mathcal{M})$ and a signer index $i \in \text{SS}$ as input and compute the mask $\text{mask}_{i, \mu, \text{SS}, \mathcal{M}}$. Also, G_1 and G_2 maintain a set setQ to record all second-round queries and maintain a table curMS , where the key of each entry is a second-round query and the entry records the set of signers that have received the query. The sets and the tables are initially empty. The rest of each game is the same as $\text{adp-TSUF}_{\text{TS}[N, T]}$.

The differences between G_0 and G_1 only lie in the algorithm CompMask . In particular, in G_1 , CompMask samples $\text{mask}_{i, \mu, \text{SS}, \mathcal{M}}$ uniformly from $\mathcal{R}_q^m$ and then programs the random oracle accordingly if SS is not contained in $\text{curMS}(\mu, \text{SS}, \mathcal{M}) \cup \text{CS}$, i.e., the mask $\text{mask}_{i, \mu, \text{SS}, \mathcal{M}}$ is not determined by other signers' masks in SS . Otherwise, since the sum of $\{\text{mask}_{j, \mu, \text{SS}, \mathcal{M}}\}_{j \in \text{SS}}$ is 0, CompMask computes $\text{mask}_{i, \mu, \text{SS}, \mathcal{M}} \leftarrow - \sum_{j \in \text{SS} \setminus \{i\}} \text{mask}_{j, \mu, \text{SS}, \mathcal{M}}$. The detailed differences are shown in Figure 11.

Denote BadSeed as the event that $\mathcal{A}$ makes a RO query $H_3(\text{seed}_{i,j}, x)$ for some $i, j \in [N]$ and x before signer i is corrupted, or $\mathcal{A}$ makes two RO queries $H_2(x)$ and $H_2(x')$ such that $H_2(x) = H_2(x')$ and $x \neq x'$. Since each $\text{seed}_{i,j}$ is uniformly sampled from $\{0, 1\}^{2\kappa}$, the probability that $\mathcal{A}$ guesses one of an honest seed is at most $N^2 \cdot 2^{-2\kappa}$ in a single RO query. Since the output of H_2 is 2κ bits, the probability that $\mathcal{A}$ finds a collision is at most $Q_h^2 \cdot 2^{-2\kappa}$. Therefore, the probability $\Pr[\text{BadSeed}] \leq Q_h(Q_h + N^2) \cdot 2^{-2\kappa}$ in either G_0 or G_1 . Since given BadSeed does not occur, G_1 provides the same view to $\mathcal{A}$ as G_0 , $\text{Adv}^{G_1}(\mathcal{A}, \kappa) \geq \text{Adv}^{G_0}(\mathcal{A}, \kappa) - \Pr[\text{BadSeed}] \geq \text{Adv}^{G_0}(\mathcal{A}, \kappa) - Q_h(Q_h + N^2) \cdot 2^{-2\kappa}$.

G_2 : Same as G_1 except that (1) we also compute the mask $\text{mask}_{i,\mu,\text{SS},\mathcal{M}}$ when signer i is corrupted even if signer i did not receive the second-round query $\mu, \text{SS}, \mathcal{M}$, and (2) we delay the programming of H_3 until the corresponding signer is corrupted. Concretely, the differences are shown in Figure 12. Similarly, given BadSeed does not occur, G_2 provides the same view to $\mathcal{A}$ as G_1 . Therefore, $\text{Adv}^{G_2}(\mathcal{A}, \kappa) \geq \text{Adv}^{G_1}(\mathcal{A}, \kappa) - \Pr[\text{BadSeed}] \geq \text{Adv}^{G_1}(\mathcal{A}, \kappa) - Q_h(Q_h + N^2) \cdot 2^{-2\kappa}$.

G_3 : Same as G_2 except we change the order of the sampling of the masks $\text{mask}_{i,\mu,\text{SS},\mathcal{M}}$ and the computation of the second-round responses $\mathbf{z}_{i,\mu,\text{SS},\mathcal{M}}$. In particular, in the algorithm $\text{CompMask}(i, \mu, \text{SS}, \mathcal{M})$, we first compute $\mathbf{z}_{i,\mu,\text{SS},\mathcal{M}}$ and then derive the corresponding mask $\text{mask}_{i,\mu,\text{SS},\mathcal{M}}$ as follows. There are two cases: (1) CompMask is invoked by the oSIGN_2 oracle, in which case $i \notin \text{CS}$; (2) CompMask is invoked by the COR , in which case $i \in \text{CS}$. For the first case, if i is not the last one in $\text{SS} \setminus \text{CS}$ such that $\mathbf{z}_{i,\mu,\text{SS},\mathcal{M}}$ is not computed yet, (equivalently, $\text{SS} \setminus (\text{CS} \cup \text{curMS}(\mu, \text{SS}, \mathcal{M})) \neq \emptyset$) we sample $\mathbf{z}_{i,\mu,\text{SS},\mathcal{M}}$ uniformly from $\mathcal{R}_q^m$. Otherwise, denoting $\text{hSet} = \text{SS} \setminus \text{CS}$ and $\text{cSet} = \text{SS} \cap \text{CS}$, since in G_2 ,

$$\begin{aligned} \sum_{j \in \text{hSet}} \mathbf{z}_j &= \sum_{j \in \text{hSet}} (\mathbf{r}_j + L_{\text{SS},j} \cdot \mathbf{T}_j \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) + \text{mask}_j) \\ &= \sum_{j \in \text{hSet}} \mathbf{r}_j + \left(\mathbf{T} - \sum_{j \in \text{cSet}} L_{\text{SS},j} \mathbf{T}_j \right) \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) \\ &\quad - \sum_{j \in \text{cSet}} \text{mask}_j, \end{aligned} \tag{5}$$

we compute

$$\begin{aligned} \mathbf{z}_{i,\mu,\text{SS},\mathcal{M}} &= \sum_{j \in \text{hSet}} \mathbf{r}_{j,\mu,\text{SS},\mathcal{M}} + \left(\mathbf{T} - \sum_{j \in \text{cSet}} L_{\text{SS},j} \mathbf{T}_j \right) \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) \\ &\quad - \sum_{j \in \text{cSet}} \text{mask}_{j,\mu,\text{SS},\mathcal{M}} - \sum_{j \in \text{hSet} \setminus \{i\}} \mathbf{z}_{j,\mu,\text{SS},\mathcal{M}}. \end{aligned} \tag{6}$$

Finally, we compute $\text{mask}_{i,\mu,\text{SS},\mathcal{M}} = \mathbf{z}_{i,\mu,\text{SS},\mathcal{M}} - L_{\text{SS},i} \mathbf{T}_i \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) - \mathbf{r}_{i,\mu,\text{SS},\mathcal{M}}$.

For the second case, the only differences are that we set $\mathbf{r}_{i,\mu,\text{SS},\mathcal{M}} = 0$ and in Equation (6), cSet is replaced by $\text{cSet} \setminus \{i\}$.⁸ The changes are also shown in Figure 12.

It is clear that the view of $\mathcal{A}$ remains the same in the first case. We now show that the view of $\mathcal{A}$ also remains the same in the second case. Note that in the second case, the value $\mathbf{z}_{i,\mu,\text{SS},\mathcal{M}}$ is never used later in the game. Therefore, we just need to show that the distribution of $\text{mask}_{i,\mu,\text{SS},\mathcal{M}}$ remains the same. If $\text{SS} \setminus (\text{CS} \cup \text{curMS}(\mu, \text{SS}, \mathcal{M})) \neq \emptyset$, $\text{mask}_{i,\mu,\text{SS},\mathcal{M}}$ is uniformly distributed over $\mathcal{R}_q^m$, which is consistent with G_2 . Otherwise,

⁸ Note in the first case, since $i \notin \text{cSet}$, we can also use $\text{cSet} \setminus \{i\}$ instead, which further reduces the distinction between the two cases.

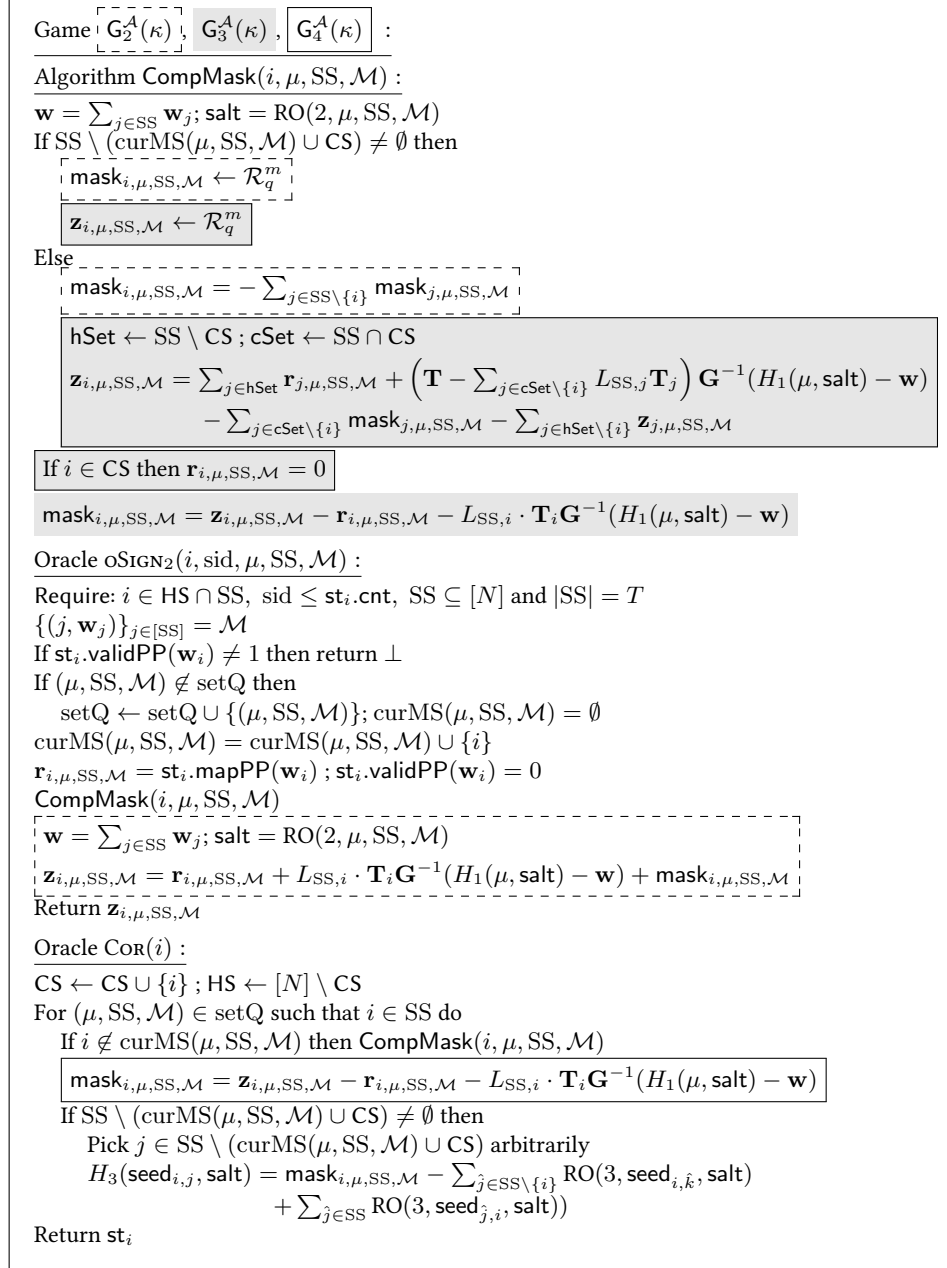


Fig. 12. The oracles oSIGN_2 and COR of the games $\mathcal{G}_2$, $\mathcal{G}_3$, and $\mathcal{G}_4$, where $\mathcal{G}_2$ only contains the dashed box, $\mathcal{G}_3$ only contains the highlighted boxes, and $\mathcal{G}_4$ only contains the solid boxes. The rest of each game is the same as $\mathcal{G}_2$.

in G_3 , by the above description, since $\mathbf{r}_{i,\mu,SS,\mathcal{M}} = 0$,

$$\begin{aligned} \text{mask}_{i,\mu,SS,\mathcal{M}} &= \mathbf{z}_{i,\mu,SS,\mathcal{M}} - \mathbf{r}_{i,\mu,SS,\mathcal{M}} - L_{SS,i} \mathbf{T}_i \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) \\ &= \sum_{j \in \text{hSet}} \mathbf{r}_{j,\mu,SS,\mathcal{M}} + \left(\mathbf{T} - \sum_{j \in \text{cSet} \setminus \{i\}} L_{SS,j} \mathbf{T}_j \right) \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) \\ &\quad - \sum_{j \in \text{cSet} \setminus \{i\}} \text{mask}_{j,\mu,SS,\mathcal{M}} - \sum_{j \in \text{hSet} \setminus \{i\}} \mathbf{z}_{j,\mu,SS,\mathcal{M}} \\ &\quad - L_{SS,i} \mathbf{T}_i \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}), \end{aligned}$$

which is consistent with Equation (5). (Note that in the above equation, $\text{hSet} = \text{hSet} \setminus \{i\}$ since $i \notin \text{hSet}$.)

Therefore, the view of $\mathcal{A}$ remains the same after the changes, which means $\text{Adv}^{G_3}(\mathcal{A}, \kappa) = \text{Adv}^{G_2}(\mathcal{A}, \kappa)$.

G_4 : Same as G_3 except that we further delay the computation of the masks in oSIGN_2 to the COR oracle. The differences are shown in Figure 12. We can delay the computation since the masks $\text{mask}_{i,\mu,SS,\mathcal{M}}$ for $i \in \text{HS}$ are not used until signer i is corrupted. It is clear that the view of $\mathcal{A}$ remains the same. Therefore, $\text{Adv}^{G_4}(\mathcal{A}, \kappa) = \text{Adv}^{G_3}(\mathcal{A}, \kappa)$.

We now construct an adversary $\mathcal{B}$ playing the adp-IUF game that runs $\mathcal{A}$ by simulating the G_4 game.

Construction of $\mathcal{B}$. After receiving $\mathbf{A}$ from the adp-IUF game, $\mathcal{B}$ initializes CS , setQ and curMS following the game G_4 . We use $\tilde{\text{st}}_i$ to denote the state of signer i maintained by $\mathcal{B}$ as in the game G_4 . Then, $\mathcal{B}$ samples the seed $\text{seed}_{i,j}$ for each $i, j \in [N]$ and runs $\mathcal{A}$ with input $\mathbf{A}$ and access to oracles $\widetilde{\text{oSIGN}}_1, \widetilde{\text{oSIGN}}_2, \widetilde{\text{RO}}, \widetilde{\text{COR}}$, which are simulated as follows.

$\widetilde{\text{RO}}(i, x)$: If $i \in \{1, 2\}$, $\mathcal{B}$ directly forwards the query to its own oracle RO . Otherwise, $\mathcal{B}$ simulates the random oracle by lazy sampling as follows. If $H_3(y) = \perp$, $\mathcal{B}$ samples $H_3(y) \leftarrow \mathcal{R}_q^m$.

$\widetilde{\text{oSIGN}}_1(i)$: Same as oSIGN_1 in G_4 except that $\mathcal{B}$ sets $\mathbf{w}_i$ by querying the oSIGN_1 oracle in the adp-IUF game, i.e., $\mathbf{w}_i \leftarrow \text{oSIGN}_1(i)$, and sets $\tilde{\text{st}}_i.\text{mapPP}(\mathbf{w}_i) = \perp$.

$\widetilde{\text{oSIGN}}_2(i, \text{sid}, \mu, \text{SS}, \mathcal{M})$: Same as oSIGN_2 in G_4 except that $\mathcal{B}$ the execution of the algorithms CompMask is changed as described below.

$\widetilde{\text{COR}}(i)$: $\mathcal{B}$ queries $\tilde{\text{st}}_i \leftarrow \text{COR}(i)$, where COR is the oracle in the adp-IUF game. Then, $\mathcal{B}$ samples $\mathbf{T}_i \leftarrow \mathcal{R}_q^{m \times \ell}$ and sets $\tilde{\text{st}}_i.\text{sk} = (\mathbf{T}_i, \{\text{seed}_{i,j}, \text{seed}_{j,i}\}_{j \in [N]})$. Then, for each $(\mu, \text{SS}, \mathcal{M}) \in \text{setQ}$ such that $i \in \text{curMS}(\mu, \text{SS}, \mathcal{M})$, $\mathcal{B}$ sets $\mathbf{r}_{i,\mu,SS,\mathcal{M}} = \tilde{\text{st}}_i.\text{mapPP}(\mathbf{w}_i)$. Finally, $\mathcal{B}$ executes $\text{COR}(i)$ as in G_4 except that $\mathcal{B}$ queries $\widetilde{\text{RO}}(x)$ instead of $\text{RO}(x)$ and returns $\tilde{\text{st}}_i$, and the execution of the algorithms CompMask is changed as described below.

$\text{CompMask}(i, \mu, \text{SS}, \mathcal{M})$: Same as G_4 except when $\text{SS} \subseteq \text{CS} \cup \text{curMS}(\mu, \text{SS}, \mathcal{M})$, $\mathcal{B}$ queries $\bar{\mathbf{z}} \leftarrow \widetilde{\text{oSIGN}}_2(\mu, \text{SS}, \mathcal{M})$ and computes $\mathbf{z}_{i,\mu,SS,\mathcal{M}} = \bar{\mathbf{z}} - \sum_{j \in \text{cSet} \setminus \{i\}} L_{SS,j} \cdot \mathbf{T}_j \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}) - \sum_{j \in \text{cSet} \setminus \{i\}} \text{mask}_{j,\mu,SS,\mathcal{M}} - \sum_{j \in \text{hSet} \setminus \{i\}} \mathbf{z}_{j,\mu,SS,\mathcal{M}}$, where $\text{cSet} = \text{SS} \cap \text{CS}$ and $\text{hSet} = \text{SS} \setminus \text{HS}$.

After $\mathcal{A}$ returns a forgery (μ^*, sig^*) , $\mathcal{B}$ outputs (μ^*, sig^*) .

Analysis of $\mathcal{B}$. We first show $\mathcal{B}$ simulates the oracles perfectly. It is clear that $\widetilde{\text{RO}}$ oracle and $\widetilde{\text{oSIGN}}_1$ are simulated perfectly. For the oracle $\widetilde{\text{oSIGN}}_2$, we just need to show that $\mathbf{z}_{i,\mu,SS,\mathcal{M}}$ is computed consistent with the game G_4 when $\text{SS} \subseteq \text{CS} \cup \text{curMS}(\mu, \text{SS}, \mathcal{M})$. By description of the oSIGN_2 oracle in the adp-IUF game,

$$\bar{\mathbf{z}} = \sum_{j \in \text{hSet}} \mathbf{r}_{j,\mu,SS,\mathcal{M}} + \mathbf{T} \mathbf{G}^{-1}(H_1(\mu, \text{salt}) - \mathbf{w}),$$

where $\{(j, \mathbf{w}_j)\}_{j \in \text{SS}} = \mathcal{M}$, $\mathbf{w} = \sum_{j \in \text{SS}} \mathbf{w}_j$ and $\mathbf{r}_{j,\mu,SS,\mathcal{M}} = \text{st}_j.\text{mapPP}(\mathbf{w}_j)$. Here st_j denotes the state of signer j in the adp-IUF game. Therefore, $\mathbf{z}_{i,\mu,SS,\mathcal{M}}$ is computed consistent with the game G_4 . For the oracle $\widetilde{\text{COR}}$, we just need to show the mask $\text{mask}_{i,\mu,SS,\mathcal{M}}$ is computed consistently with the game G_4 . If $i \notin \text{curMS}(\mu, \text{SS}, \mathcal{M})$, similar to the oracle $\widetilde{\text{oSIGN}}_2$, the execution of CompMask is simulated perfectly. If $i \in \text{curMS}(\mu, \text{SS}, \mathcal{M})$, by the execution,

$\tilde{\text{st}}.\text{validPP}(\mathbf{w}_i) \neq \perp$, where $\{(j, \mathbf{w}_j)\}_{j \in \text{SS}} = \mathcal{M}$. Therefore, $\mathbf{r}_{i,\mu,\text{SS},\mathcal{M}} = \tilde{\text{st}}.\text{mapPP}(\mathbf{w}_i)$ is consistent with the game G_4 , which implies the computation of $\text{mask}_{i,\mu,\text{SS},\mathcal{M}}$ is also consistent with the game G_4 .

Therefore, $\mathcal{B}$ simulates the G_4 game perfectly. The number of oSIGN_1 queries made by $\mathcal{B}$ is at most Q_s , and the number of random oracle queries made by $\mathcal{B}$ is at most $Q_h + Q_s$.

It is left to show that if $\mathcal{A}$ wins the G_4 simulated by $\mathcal{B}$, $\mathcal{B}$ wins the adp-IUF game. Since $\mathcal{B}$ makes a query to $\text{oSIGN}_2(\mu, \text{SS}, \mathcal{M})$ if and only if $\text{SS} \subseteq \text{CS} \cup \text{curMS}(\mu, \text{SS}, \mathcal{M})$, the set TrivQ defined in adp-IUF is identical to the set TrivQ defined in G_4 , which implies the statement. $\square$

Acknowledgments. KJ and HW thank Divesh Aggarwal for hosting their visits at NUS and CQT, where part of this work was done. KJ is supported by NRF investigatorship grant NRF-NRFI09-0005) and the Breakthrough Plan of the Ministry of Education, China (No. JYB2025XDXM114). AI tools were used solely for grammar correction and minor language editing. All technical content, results, and proofs are the authors' own.

References

- AGHS13. Shweta Agrawal, Craig Gentry, Shai Halevi, and Amit Sahai. Discrete Gaussian leftover hash lemma over infinite domains. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part I*, volume 8269 of *LNCS*, pages 97–116. Springer, Berlin, Heidelberg, December 2013.
- ALLW25. Martin R. Albrecht, Russell W. F. Lai, Oleksandra Lapiha, and Ivy K. Y. Woo. Partial lattice trapdoors: How to split lattice trapdoors, literally. In *ASIACRYPT*, 2025.
- ASY22. Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-optimal lattice-based threshold signatures, revisited. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *ICALP 2022*, volume 229 of *LIPIcs*, pages 8:1–8:20. Schloss Dagstuhl, July 2022.
- BCdP⁺25. Giacomo Borin, Sofia Celi, Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Threshold signatures reloaded: ML-DSA and enhanced raccoon with identifiable aborts. *Cryptology ePrint Archive*, Paper 2025/1166, 2025.
- BCL⁺26. Renas Bacho, Yanbo Chen, Julian Loss, Stefano Tessaro, and Chenzhi Zhu. Adaptively secure partially non-interactive threshold schnorr signatures in the AGM. In *EUROCRYPT*, 2026.
- BdCE⁺25. Alexander Bienstock, Leo de Castro, Daniel Escudero, Antigoni Polychroniadou, and Akira Takahashi. Efficient, scalable threshold ML-DSA signatures: An MPC approach. *Cryptology ePrint Archive*, Paper 2025/1163, 2025.
- BDLR25a. Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Adaptively secure three-round threshold schnorr signatures from DDH. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 390–422. Springer, Cham, August 2025.
- BDLR25b. Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Glacius: Threshold schnorr signatures from DDH with full adaptive security. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 304–334. Springer, Cham, May 2025.
- BGG⁺18. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 565–596. Springer, Cham, August 2018.
- BGGK17. Dan Boneh, Rosario Gennaro, Steven Goldfeder, and Sam Kim. A lattice-based universal thresholdizer for cryptographic systems. *Cryptology ePrint Archive*, Report 2017/251, 2017.
- BKL⁺25. Cecilia Boschini, Darya Kaviani, Russell W. F. Lai, Giulio Malavolta, Akira Takahashi, and Mehdi Tibouchi. Ringtail: Practical two-round threshold signatures from learning with errors. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 149–164. IEEE Computer Society Press, May 2025.
- BL22. Renas Bacho and Julian Loss. On the adaptive security of the threshold BLS signature scheme. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 193–207. ACM Press, November 2022.
- BLT⁺24. Renas Bacho, Julian Loss, Stefano Tessaro, Benedikt Wagner, and Chenzhi Zhu. Twinkle: Threshold signatures from DDH with full adaptive security. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 429–459. Springer, Cham, May 2024.
- Bol03. Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003*, volume 2567 of *LNCS*, pages 31–46. Springer, Berlin, Heidelberg, January 2003.
- BTZ22. Mihir Bellare, Stefano Tessaro, and Chenzhi Zhu. Stronger security for non-interactive threshold signatures: BLS and FROST. *Cryptology ePrint Archive*, Report 2022/833, 2022.

- CATZ24. Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Partially non-interactive two-round lattice-based threshold signatures. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 268–302. Springer, Singapore, December 2024.
- CGM19. Yilei Chen, Nicholas Genise, and Pratyay Mukherjee. Approximate trapdoors for lattices and smaller hash-and-sign signatures. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 3–32. Springer, Cham, December 2019.
- Che25a. Yanbo Chen. Dazzle: Improved adaptive threshold signatures from DDH. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part III*, volume 15676 of *LNCS*, pages 233–261. Springer, Cham, May 2025.
- Che25b. Yanbo Chen. Round-efficient adaptively secure threshold signatures with rewinding. *IACR Communications in Cryptology*, 2(2), 2025.
- Che26. Yanbo Chen. Partially non-interactive two-round threshold and multi-signatures with tighter and adaptive security. Cryptology ePrint Archive, Paper 2026/373, 2026.
- CKGW22. Deirdre Connolly, Chelsea Komlo, Ian Goldberg, and Christopher A. Wood. Two-Round Threshold Schnorr Signatures with FROST. Internet-Draft draft-irtf-cfrg-frost-10, Internet Engineering Task Force, September 2022. Work in Progress.
- CKK⁺25. Elizabeth C. Crites, Jonathan Katz, Chelsea Komlo, Stefano Tessaro, and Chenzhi Zhu. On the adaptive security of FROST. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 480–511. Springer, Cham, August 2025.
- DDFY94. Alfredo De Santis, Yvo Desmedt, Yair Frankel, and Moti Yung. How to share a function securely. In *26th ACM STOC*, pages 522–533. ACM Press, May 1994.
- Des88. Yvo Desmedt. Society and group oriented cryptography: A new concept. In Carl Pomerance, editor, *CRYPTO’87*, volume 293 of *LNCS*, pages 120–127. Springer, Berlin, Heidelberg, August 1988.
- DF90. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *CRYPTO’89*, volume 435 of *LNCS*, pages 307–315. Springer, New York, August 1990.
- DKM⁺24. Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 219–248. Springer, Cham, May 2024.
- dPENP25. Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Simple and efficient lattice threshold signatures with identifiable aborts. Cryptology ePrint Archive, Paper 2025/871, 2025.
- DR24. Sourav Das and Ling Ren. Adaptively secure BLS threshold signatures from DDH and co-CDH. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 251–284. Springer, Cham, August 2024.
- EEN⁺24. Muhammed F. Esgin, Thomas Espitau, Guilhem Niot, Thomas Prest, Amin Sakzad, and Ron Steinfeld. Plover: Masking-friendly hash-and-sign lattice signatures. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VII*, volume 14657 of *LNCS*, pages 316–345. Springer, Cham, May 2024.
- EKT24. Thomas Espitau, Shuichi Katsumata, and Kaoru Takemure. Two-round threshold signature from algebraic one-more learning with errors. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 387–424. Springer, Cham, August 2024.
- ENP24. Thomas Espitau, Guilhem Niot, and Thomas Prest. Flood and submerge: Distributed key generation and robust threshold signature from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 425–458. Springer, Cham, August 2024.
- EY24. Ehsan Ebrahimi and Anshu Yadav. Strongly secure universal thresholdizer. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part III*, volume 15486 of *LNCS*, pages 207–239. Springer, Singapore, December 2024.
- FNR25. Sebastian Faller, Guilhem Niot, and Michael Reichle. Lattice-based threshold blind signatures. Cryptology ePrint Archive, Paper 2025/1566, 2025.
- GCRS25. Paul Gerhart, Davide Li Calsi, Luigi Russo, and Dominique Schröder. Fully-adaptive two-round threshold schnorr signatures from DDH. Cryptology ePrint Archive, Paper 2025/1478, 2025.
- GKS24. Kamil Doruk Gür, Jonathan Katz, and Tjerand Silde. Two-round threshold lattice-based signatures from threshold homomorphic encryption. In Markku-Juhani Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, pages 266–300. Springer, Cham, June 2024.
- GMPW20. Nicholas Genise, Daniele Micciancio, Chris Peikert, and Michael Walter. Improved discrete gaussian and subgaussian analysis for lattice cryptography. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part I*, volume 12110 of *LNCS*, pages 623–651. Springer, Cham, May 2020.
- GPV08. Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 197–206. ACM Press, May 2008.
- GRJK07. Rosario Gennaro, Tal Rabin, Stanislaw Jarecki, and Hugo Krawczyk. Erratum: Robust and efficient sharing of RSA functions. *Journal of Cryptology*, 20(3):393, July 2007.

- GTJ⁺26. Liming Gao, Guofeng Tang, Dingding Jia, Yijian Liu, Bingqian Liu, Xianhui Lu, Kunpeng Wang, and Yongjian Yin. TalonG: Bandwidth-efficient two-round threshold signatures from lattices. In *EUROCRYPT*, 2026.
- JTWZ26. Kaijie Jiang, Stefano Tessaro, Hoeteck Wee, and Chenzhi Zhu. Tweed: Adaptively secure lattice-based two-round threshold signatures. In *EUROCRYPT*, 2026.
- KRT24. Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, Cham, August 2024.
- Mak22. Nikolaos Makriyannis. On the classic protocol for MPC schnorr signatures. Cryptology ePrint Archive, Paper 2022/1332, 2022.
- MP12. Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 700–718. Springer, Berlin, Heidelberg, April 2012.
- MP13. Daniele Micciancio and Chris Peikert. Hardness of SIS and LWE with small parameters. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 21–39. Springer, Berlin, Heidelberg, August 2013.
- Natnt. National Institute of Standards and Technology. Multi-Party Threshold Cryptography, 2018–Present. <https://csrc.nist.gov/Projects/threshold-cryptography>.
- Pei06. Chris Peikert. On error correction in the exponent. In Shai Halevi and Tal Rabin, editors, *TCC 2006*, volume 3876 of *LNCS*, pages 167–183. Springer, Berlin, Heidelberg, March 2006.
- Pei15. Chris Peikert. A decade of lattice cryptography. Cryptology ePrint Archive, Report 2015/939, 2015.
- PKN⁺25. Rafaël Del Pino, Shuichi Katsumata, Guilhem Niot, Michael Reichle, and Kaoru Takemure. Unmasking TRaccoon: A lattice-based threshold signature with an efficient identifiable abort protocol. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 423–456. Springer, Cham, August 2025.
- Ros20. Mélissa Rossi. *Extended security of lattice-based cryptography*. PhD thesis, Université Paris sciences et lettres, 2020.
- Sho00. Victor Shoup. Practical threshold signatures. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 207–220. Springer, Berlin, Heidelberg, May 2000.
- TT15. Katsuyuki Takashima and Atsushi Takayasu. Tighter security for efficient lattice cryptography via the Rényi divergence of optimized orders. In Man Ho Au and Atsuko Miyaji, editors, *ProvSec 2015*, volume 9451 of *LNCS*, pages 412–431. Springer, Cham, November 2015.
- YJW23. Yang Yu, Huiwen Jia, and Xiaoyun Wang. Compact lattice gadget and its applications to hash-and-sign signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 390–420. Springer, Cham, August 2023.
- ZT25. Chenzhi Zhu and Stefano Tessaro. The algebraic one-more MISIS problem and applications to threshold signatures. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part I*, volume 16000 of *LNCS*, pages 548–581. Springer, Cham, August 2025.

A Proof of Lemma 6

We first show the statement on the distribution $\mathcal{D}$. Since the distribution of $(\mathbf{r}_1, \dots, \mathbf{r}_t)^\top$ is $\mathcal{D}_{(\mathbf{P} \otimes \mathbf{I}_m) \cdot \Lambda^t + \mathbf{c}, \sigma_r}$. By Lemma 5, we just need to show that:

- $\phi_M(\mathbf{P} \otimes \mathbf{I}_m)$ has full-row-rank, and $\ker(\mathbf{P} \otimes \mathbf{I}_m)$ is a Λ^t -subspace.
- $\eta_{\varepsilon'}(\Lambda^t \cap \ker(\mathbf{P} \otimes \mathbf{I}_m)) \leq \sigma_r$, where $\varepsilon' = (1 + \varepsilon)^{t-k-1} - 1$.

By the definition of $\mathbf{P}$, $\phi_M(\mathbf{P} \otimes \mathbf{I}_m)$ has full-row-rank. In order to show $\ker(\mathbf{P} \otimes \mathbf{I}_m)$ is a Λ^t -subspace, we just need to show $\text{Span}(\Lambda^t \cap \ker(\mathbf{P} \otimes \mathbf{I}_m)) = \ker(\mathbf{P} \otimes \mathbf{I}_m)$. For any $t \geq j > k+1$, we have $\mathbf{v}_j := \mathbf{e}_{k+1} - \mathbf{e}_j \in \ker(\mathbf{P}) \subseteq \mathcal{R}^t$. Let Λ_0 be the lattice generated by $(\mathbf{v}_j)_{j \in [k+2, t]}$, then we have $\Lambda_0 \otimes \Lambda \subseteq \Lambda^t \cap \ker(\mathbf{P} \otimes \mathbf{I}_m)$. Note that $\ker(\mathbf{P} \otimes \mathbf{I}_m) = \text{Span}(\Lambda_0 \otimes \Lambda) \subseteq \text{Span}(\Lambda^t \cap \ker(\mathbf{P} \otimes \mathbf{I}_m)) \subseteq \ker(\mathbf{P} \otimes \mathbf{I}_m)$, thus $\text{Span}(\Lambda^t \cap \ker(\mathbf{P} \otimes \mathbf{I}_m)) = \ker(\mathbf{P} \otimes \mathbf{I}_m)$.

For the second condition, we just need to show $\eta_{\varepsilon'}(\Lambda^t \cap \ker(\mathbf{P} \otimes \mathbf{I}_m)) \leq \eta_{\varepsilon'}(\Lambda_0 \otimes \Lambda) \leq \sigma_r$, note that $\Lambda_0 \otimes \Lambda = \sum_{j=k+2}^t \mathbf{v}_j \otimes \Lambda$ and $\tilde{\mathbf{v}}_i \otimes \Lambda$ is the projection of $\mathbf{v}_i \otimes \Lambda$ orthogonal to $\text{Span}(\mathbf{v}_{k+2} \otimes \Lambda + \dots + \mathbf{v}_{i-1} \otimes \Lambda)$. By applying Lemma 12 $t-k-1$ times, we have $\eta_{\varepsilon'}(\Lambda_0 \otimes \Lambda) \leq \max_{j \in [k+2, t]} \eta_\varepsilon(\tilde{\mathbf{v}}_j \otimes \Lambda)$. Note that for any $j \in [k+2, t]$ $\tilde{\mathbf{v}}_j = \frac{1}{j-k-1} \cdot \sum_{i=k+1}^{j-1} \mathbf{e}_i - \mathbf{e}_j$, thus we have $\|\tilde{\mathbf{v}}_j\|_1 = 2$. By Lemma 13, $\eta_\varepsilon(\tilde{\mathbf{v}}_j \otimes \Lambda) \leq 2 \cdot \eta_\varepsilon(\Lambda) \leq \sigma_r$, which implies $\eta_{\varepsilon'}(\Lambda^t \cap \ker(\mathbf{P} \otimes \mathbf{I}_m)) \leq \sigma_r$.

For the statement on $\mathcal{D}_1$ following the similar proof as above, where we used $(\mathbf{P}_1 \otimes \mathbf{I}_m)\Lambda^t = \Lambda^k$.

For $\mathcal{D}_2|_{\mathbf{x}_0}$, we just need to show that assuming $(\mathbf{y}_1, \mathbf{x})^\top$ is sampled from $\mathcal{D}_{(\mathbf{P} \otimes \mathbf{I}_m)\Lambda^t + \mathbf{c}, \Sigma \otimes \mathbf{I}_m}$, the distribution of $\mathbf{y}_1$ conditioning on $\mathbf{x} = \mathbf{x}_0$ is identical to the target distribution for any $\mathbf{x}_0 \in \Lambda^k + \mathbf{a}_1$. Firstly, we know the support of $\mathbf{y}_1$ is $\text{Supp} := \{\mathbf{y}_1 : (\mathbf{y}_1, \mathbf{x}_0)^\top \in \Lambda^t + \mathbf{c}\} = \Lambda + \mathbf{s}_1$, since $(\mathbf{P} \otimes \mathbf{I}_m)\Lambda^t = \Lambda^k$. It is left to show the probability of $\mathbf{y}_1$ conditioning on $\mathbf{x} = \mathbf{x}_0$. The probability of $\begin{pmatrix} \mathbf{y}_1 \\ \mathbf{x}_0 \end{pmatrix}$ is proportional to

$$\begin{aligned} \rho_{\Sigma \otimes \mathbf{I}_m}((\mathbf{y}_1, \mathbf{x}_0)^\top) &= \exp(-\pi \langle (\mathbf{y}_1, \mathbf{x}_0)^\top, (\Sigma^{-1} \otimes \mathbf{I}_m) \cdot (\mathbf{y}_1, \mathbf{x}_0)^\top \rangle) \\ &= \exp(-\pi (\langle \mathbf{y}_1, V_{11} \mathbf{y}_1 \rangle - 2 \langle \mathbf{y}_1, -(\mathbf{V}_{21} \otimes \mathbf{I}_m) \mathbf{x}_0 \rangle + \langle \mathbf{x}_0, (\mathbf{V}_{22} \otimes \mathbf{I}_m) \mathbf{x}_0 \rangle)) \\ &= \exp(-\pi (\langle \mathbf{y}_1 + V_{11}^{-1}(\mathbf{V}_{21} \otimes \mathbf{I}_m) \mathbf{x}_0, V_{11}(\mathbf{y}_1 + V_{11}^{-1}(\mathbf{V}_{21} \otimes \mathbf{I}_m) \mathbf{x}_0) \rangle \\ &\quad + \langle \mathbf{x}_0, (\mathbf{V}_{22} \otimes \mathbf{I}_m) \mathbf{x}_0 \rangle - \langle V_{11}^{-1}(\mathbf{V}_{21} \otimes \mathbf{I}_m) \mathbf{x}_0, V_{11}^{-1}(\mathbf{V}_{21} \otimes \mathbf{I}_m) \mathbf{x}_0 \rangle)) \end{aligned}$$

Thus the probability of $\mathbf{y}_1$ conditioning on $\mathbf{x} = \mathbf{x}_0$ is proportional to

$$\exp(-\pi (\langle \mathbf{y}_1 + V_{11}^{-1}(\mathbf{V}_{21} \otimes \mathbf{I}_m) \mathbf{x}_0, V_{11}(\mathbf{y}_1 + V_{11}^{-1}(\mathbf{V}_{21} \otimes \mathbf{I}_m) \mathbf{x}_0) \rangle))$$

which implies the statement. $\square$

Lemma 12 ([MP13, Lemma 2.6]). For any lattice $\Lambda_1, \Lambda_2 \subseteq \mathcal{R}^m$, let $\tilde{\Lambda}_2$ be the projection of Λ_2 orthogonal to $\text{Span}(\Lambda_1)$, and we have

$$\eta_\varepsilon(\Lambda_1 + \Lambda_2) \leq \max\{\eta_{\varepsilon_1}(\Lambda_1), \eta_{\varepsilon_2}(\tilde{\Lambda}_2)\},$$

where $\varepsilon = (1 + \varepsilon_1)(1 + \varepsilon_2) - 1$

Lemma 13 ([CATZ24, Lemma 13]). For any real value $\varepsilon > 0$, a $\mathcal{R}$ -lattice $\Lambda \subseteq \mathcal{R}^m$ and a vector $\mathbf{v} \in K^n$, we have

$$\eta_\varepsilon(\mathbf{v} \otimes \Lambda) \leq \sqrt{\sigma_{\max}(\mathbf{v}^\dagger \mathbf{v})} \cdot \eta_\varepsilon(\Lambda).$$

Moreover, we have $\sqrt{\sigma_{\max}(\mathbf{v}^\dagger \mathbf{v})} \leq \|\mathbf{v}\|_1$.